

Тема 1. Софтуерни Технологии (Software Engineering) – основни понятия. Жизнен цикъл на програмното осигуряване. Модели на ЖЦ

дисциплина: Технология на софтуерното
производство

лектор: доц. В.Божикова

Какво разглежда тази тема:

- “Software Engineering” – определение и превод.
Значение на дисциплината Софтуерни Технологии.
- Жизнен цикъл на програмното осигуряване. Модели на жизнения цикъл
- Класификация на моделите
- Пълни Едномерни Модели
- Пълни Многомерни Модели
- Частични Модели
- Пълен еволюционен модел
- Пълен спирален модел
- Някои съвременни модели: V-образен модел, USDP (Unified Software Development Process)

“Software Engineering” – определение и превод

- Понятието “Software Engineering” – въведено от Наур на една конференция на НАТО през 1969г, свързана със «софтуерната криза».
- „Софтуерната криза“ – свързана с появилите се компютри от т.н. 3-то поколение. Нивото на софтуера не може да отговори на новите компютърни мощности. Така възниква естествената необходимост от специална дисциплина, която да се опита да помогне за преодоляването на софтуерната криза.

“Software Engineering” – определение и превод

- Превод на понятието “Software Engineering” – според Нели Манева и Аврам Ескенази ([1], [2]) -
«Софтуерни Технологии».
- Друг често използван превод:
«Софтуерно Инженерство»
- **Още: «Инженеринг на програмното осигуряване» и др.**

“Software Engineering” – множество дефиниции

- Д1. “Software Engineering” е установяване и използване на здрави **принципи за разработване**, с цел по **икономичен начин да се произведе софтуер**, който е **надежден и функционира ефективно** върху реална машина (дефиниция на Наур).
- Д2. Според IEEE ([Institute of Electrical and Electronics Engineers](#)), “Software Engineering” е **систематичен подход към разработване, експлоатиране, съпровождане и изваждане от експлоатация на софтуера**.

“Software Engineering” – множество дефиниции

- Д3. Н.Манева и А.Ескенази, едни от малкото автори на учебници в тази област допълват това определение по следния начин: “Software Engineering” («Софтуерни Технологии») – това е систематичен подход към разработване на **качествен софтуер**, както и към неговото предлагане на пазара, експлоатация и съпровождане.
- Д4. Според Sommerville, “Софтуерни технологии” е **инженерна дисциплината, която обхваща всички аспекти на създаването на програмния продукт (ПП)** - от началния стадий, свързан с дефиниране на изискванията към софтуера (обект на разработване), през неговото разработване, експлоатиране и съпровождане, до неговото изваждане от употреба.

В заключение:

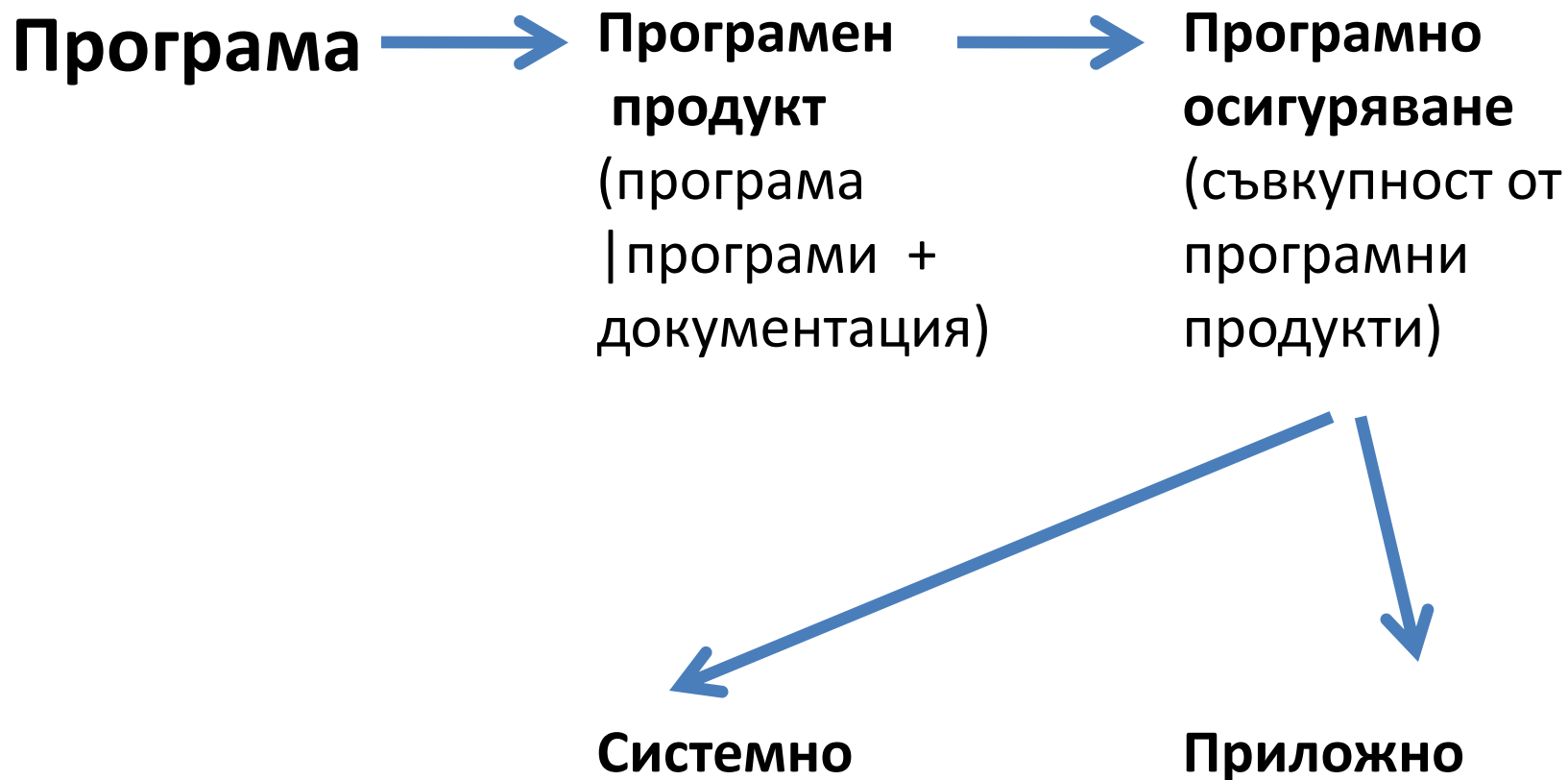
- Д5. “Софтуерни технологии” е инженерна дисциплината, която обхваща всички аспекти на създаването на качествен програмен продукт - от началния стадий, свързан с дефиниране на изискванията към софтуера (обект на разработване), през неговото разработване, експлоатиране, съпровождане, **предлагане на пазара**, до неговото изваждане от употреба.
- В това определение присъстват 3 ключови думи:

- **Инженерна дисциплината:** Софтуерните инженери прилагат теоретични знания, методи и средства, опитвайки се да намерят решение на поставената задача («инженер» в по-широк смисъл, това са всички тези специалисти, които са включени в създаването на ПП). Те търсят решение на поставената задача, отчитайки клаузите на договора (работят в условията на организационните, финансовите и времевите ограничения на сключените договори).
- **Обхваща всички аспекти на създаването на качествен програмен продукт:** не само проектиране и програмиране на програмни системи, но и такива специфични аспекти на създаването на ПП, като управление на софтуерни проекти, управление на качеството на ПП, разработване на средства, методи и инструментални средства, необходими за създаване и еволюция на програмни приложения.
- Какво се разбира под **качествен софтуер**? За да бъде един софтуер качествен, той следва да притежава следните, често противоречиви характеристики [Нели Манева]: лесно съпровождане, надеждност, ефективност, да бъде с удобен и лесен за усвояване потребителски интерфейс, да има минимална цена.

Програма - Програмен продукт (ПП) -> Програмно осигуряване (ПО)

- **Програмният продукт | софтуерният продукт (*software*)** - това е предметът на дисциплината «Софтуерни Технологии». Това е компютърна програма или съвкупност от взаимодействащи компютърни програми, записани върху технически носител и придружени със съответна документация ([1],[2]).
- **Програма** – последователност от инструкции към компютъра, които след декодиране от последния, водят до решаването от страна на компютъра на дадена задача ([1],[2]).
- **Програмното осигуряване (ПО)**, съгласно БДС, е **съвкупност от програмни продукти**. То бива **приложно** – за решаване на конкретни практически проблеми и **системно** – за управление функционирането на изчислителната машина и ресурсите и.

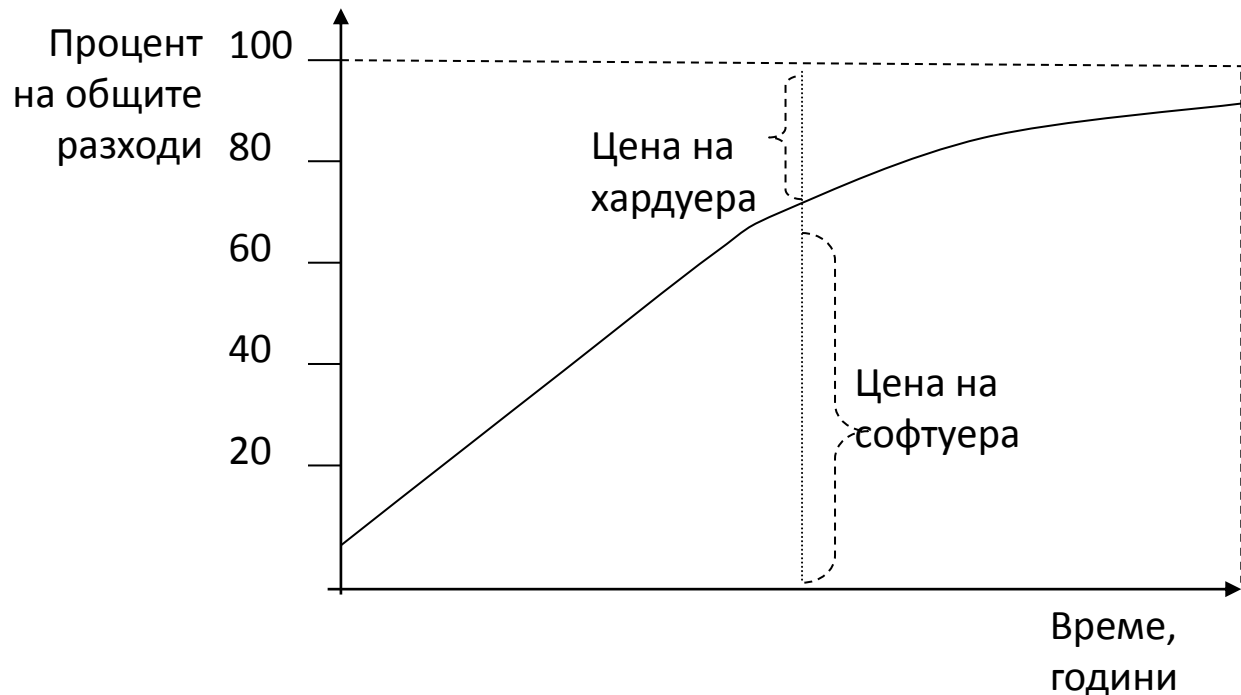
Програма -> Програмен продукт (ПП) -> Програмно осигуряване (ПО)



Значение на дисциплината Софтуерни Технологии

- Нараства! Защо?

1. Софтуерът е вече **определящ фактор** (60 - 70% от стойността) за **цената** на съвременните компютърни системи:

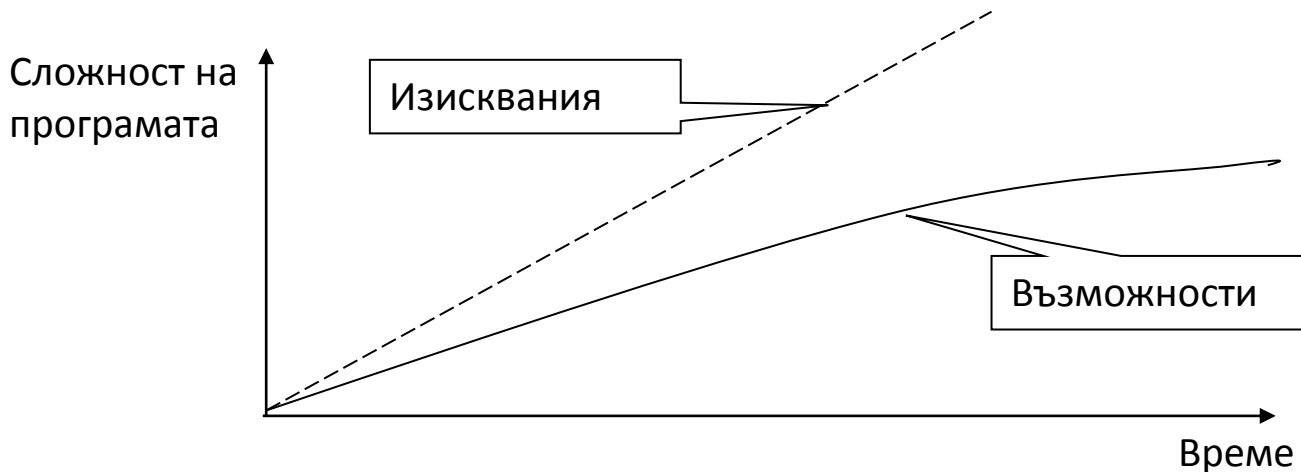


Значение на дисциплината Софтуерни Технологии

- **Нараства! Защо?**

2. Софтуерът е масов продукт и масова дейност:
наблюдава се недостиг на специалисти и глад за софтуер.

**3. Съществуват неразрешими противоречия
между изисквания и възможности**



Значение на дисциплината

Софтуерни Технологии

- **Нараства! Защо?**

4. Софтуерът и неговата разработка са рискови:

— Поради нарастващата сложност на ПП. Имаме:

- **Прости ПП** (по-малко от 1000 оператора , пишат се за време шест месеца и от един програмист, нямат връзка с други програми или системи).
- **ПП със средна сложност:** до 10000 оператора; пишат се от няколко програмиста за около 2 години; имат малко на брой или нямат връзки с други системи; обикновено съдържат от 10 до 100 модула.

- **ПП с голяма сложност:** до 100000 оператора; от 5 до 20 програмиста за около 2-3 години; съдържат няколко подсистеми; взаимодействат често с други системи; от 100 до 1000 модула.
- **ПП почти невъзможни за изпълнение:** около 10^6 оператора; пишат се от група от 100 до 1000 програмиста за няколко години; изискват привличането за дълъг период от време на хора извън основната група за проектиране; обикновено се състоят от няколко главни подсистеми със сложно взаимодействие. Такова сложно взаимодействие има и с други системи; съдържа около 1000 – 10000 модула; често се появяват и допълнителни сложности като работа в реално време, телекомуникация, многозначност.

- **направо абсурдни (по сложност) програми:**
 - имат от 10^8 и повече оператора;
 - разработват се от повече от 1000 програмиста за време от 10 и повече години;
 - почти винаги включват работа в реално време, телекомуникации и многозадачност;
 - много често отработват критични ситуации - управление на въздушен трафик, противовъздушна защита и др.;
 - обикновено имат извънредно високи изисквания за надеждност.

– **Поради спецификата на софтуера като продукт:**

Софтуерния продукт съществено се отличава от материалните продукти:

- **Абстрактност**
- **Уникалност**

– **Поради спецификата на софтуерното производство като производство:**

Софтуерното производство съществено се отличава от материалното производство

- **Мултидисциплинарно**
- **Творчески, евристичен процес, не толкова научен**
- **Софтуерът, който ще разработвате е средство, а не цел**

SWEBOK

- SWEBOK – Software Engineering Body of Knowledge (Сбор от знания по програмно Инженерство)
- SWEBOK – това е отворен проект, реализиран с поддръжката на международния комитет IEEE, започвайки от 1998.
- В SWEBOK е определен набор от знания и препоръчвани практики за разработване на ПП.
- Последната версия (SWEBOK V3) е пусната през 2013



Жизнен цикъл (ЖЦ) на софтуера (Software Life Cycle)

- Д1. Жизненият цикъл на софтуера **обхваща целия период на неговото създаване и използване, от неговото начало** (моментът на възникване на идеята за създаването му) **до неговия физически край** (моментът, в който се преустановява използването на последното копие на ПП).
 - ПП има и **логически край** – ПП се използва такъв какъвто е, без да се преработва, осъвършенства или пренася на нова платформа , не се изискват никакви усилия за съпровождането му.
- Вместо «жизнен цикъл» на софтуера (**software life cycle**), Sommerville използва понятието «процес на разработване на софтуер» (**software development process - SDP**), тоест тези понятия са **синоними**.

Жизнен цикъл (ЖЦ) на софтуера (Software Life Cycle)

- Д2. Sommerville дефинира ЖЦ (процесът на разработка на софтуера - software development process) като «**кохерентно (свързано) множество от дейности, изпълнявани при разработването на софтуерна система**».

- Т.е, това «**кохерентно (свързано) множество от дейности, изпълнявани при разработването на софтуерна система**», може да включва в частност **следните дейности:**

1. Спецификация на изискванията
2. Разработка
3. Валидация
4. Развитие

**Не съществува “идеален” ЖЦ
(SDP)!!!**

Модел на ЖЦ на софтуера

Това е едно:

- абстрактно представяне на даден процес на разработване на софтуерния продукт;
- лаконично, сбито и информативно представяне на даден процес на разработване на софтуерния продукт.

Налице е разнообразие от «**модели на ЖЦ**» на **софтуера**. Защо? Защото процесът на създаване на ПО не е еднороден.

Всеки от моделите на ЖЦ има своите силни и слаби страни и е подходящ в различни ситуации и видове проекти.

Моделирането на ЖЦ има редица технологични ползи, свързани с отговор на следните въпроси:

- какъв тип спецификации и техники на програмиране са препоръчителни?
- какви метрики да се ползват за опр. на качеството на разработвания продукт?
- възможно ли е ползването единствено на обектно-ориентиран подход от началото на ЖЦ до завършване на тестването на ПП?
- И др.

Елементи на моделите на ЖЦ на софтуера

Говорейки за модел на ЖЦ, трябва да разграничим **фази** и видове **дейности**.

- **Фаза** (phase) – това е определен етап на процеса на разработване на софтуера, имащ начало, край и изходен резултат. Например, фаза на проектиране, фаза на разработка на изискванията и т.н. Фазите следват линейно една след друга.

Модел на Жизнен цикъл на софтуера

- **Вид дейност** (activity) – това е определен тип работа, изпълнявана в процеса на разработване на ПО. Различните видове дейности често изискват различни професионални навици и се изпълняват от различни специалисти. Например, управление на проекта се изпълнява от мениджър на проекта, кодиране – от програмист, тестване – тестер. Има видове дейности, които могат да се изпълнят от едни и същи специалисти – например, кодиране и проектиране (особено за малки проекти) често се изпълняват от едни и същи хора.

Основни фази (етапи) на Жизнения цикъл на софтуера

- Независимо от голямото разнообразие от съществуващи модели на ЖЦ, основните фази (етапи), съставлящи цикъла на разработване (жизнен цикъл) на софтуера са:
- **Разработка на Изискванията към софтуера.** Тази фаза включва дейности по **Извличане, Анализ и Специфициране** (описание) на изискванията към софтуера (Software Requirement): определят се, анализират и описват очакванията на крайния потребител от готовия програмен продукт. Този етап завършва с изготвяне на задание за проектиране – това е така наречената спецификация (документ) на изискванията към софтуера (**Software Requirement Specification** или **Software Requirement Document**) или „**техническо задание**” | „**технико-икономическо задание**”, съдържащо уточнените изисквания на потребителя към системата. Разработката на изискванията (Соммервил) не е тривиална задача и се провежда по правило, по пътя на обсъждане на виждането на възложителя на системата с бъдещите разработчици.

Основни фази (етапи) на Жизнения цикъл на софтуера

- **Проектиране** - Проектиране на структурата на софтуера, която да реализира специфицираните през предната фаза изисквания; завършва със **спецификация за реализация** (Software Implementation Specification), която описва как продуктът ще постигне целта си (формулирана през фазата на изискванията). Създадената спецификация за реализация = проект за програмиране се дава на програмистите за реализация.
- **Разработка (програмиране, т.е кодиране):** създаването на работеща програмна система.

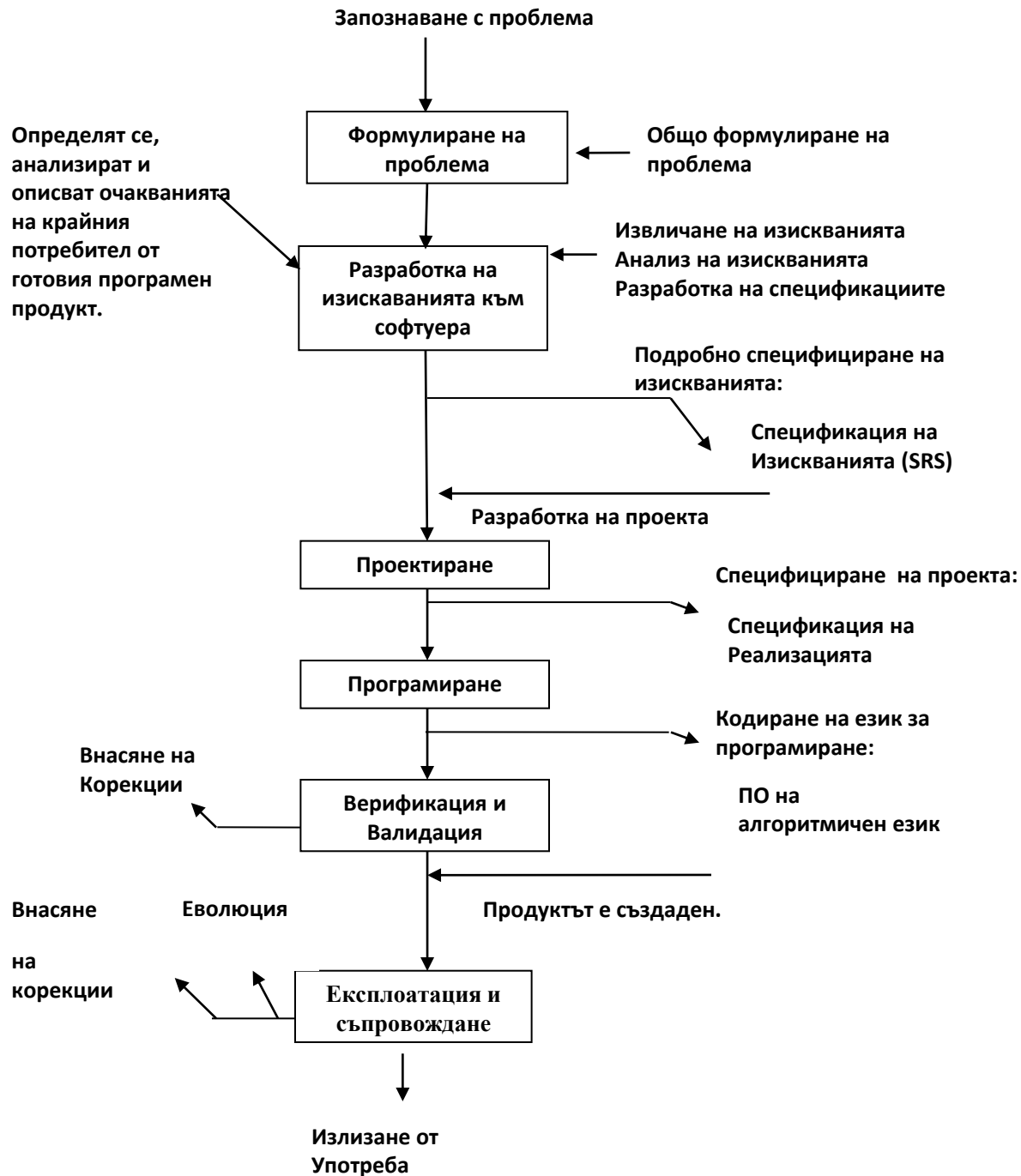
Основни фази (етапи) на Жизнения цикъл на софтуера

- **Тестване (Валидация) и Настройка (Верификация).** Програмите тестват и настройват.
 - **Верификация** - да даде отговор на въпроса дали програмата работи коректно ("Are we building the product right?").
 - **Валидация** (проверка дали софтуера е това, което клиента иска ("Are we building the right product?").

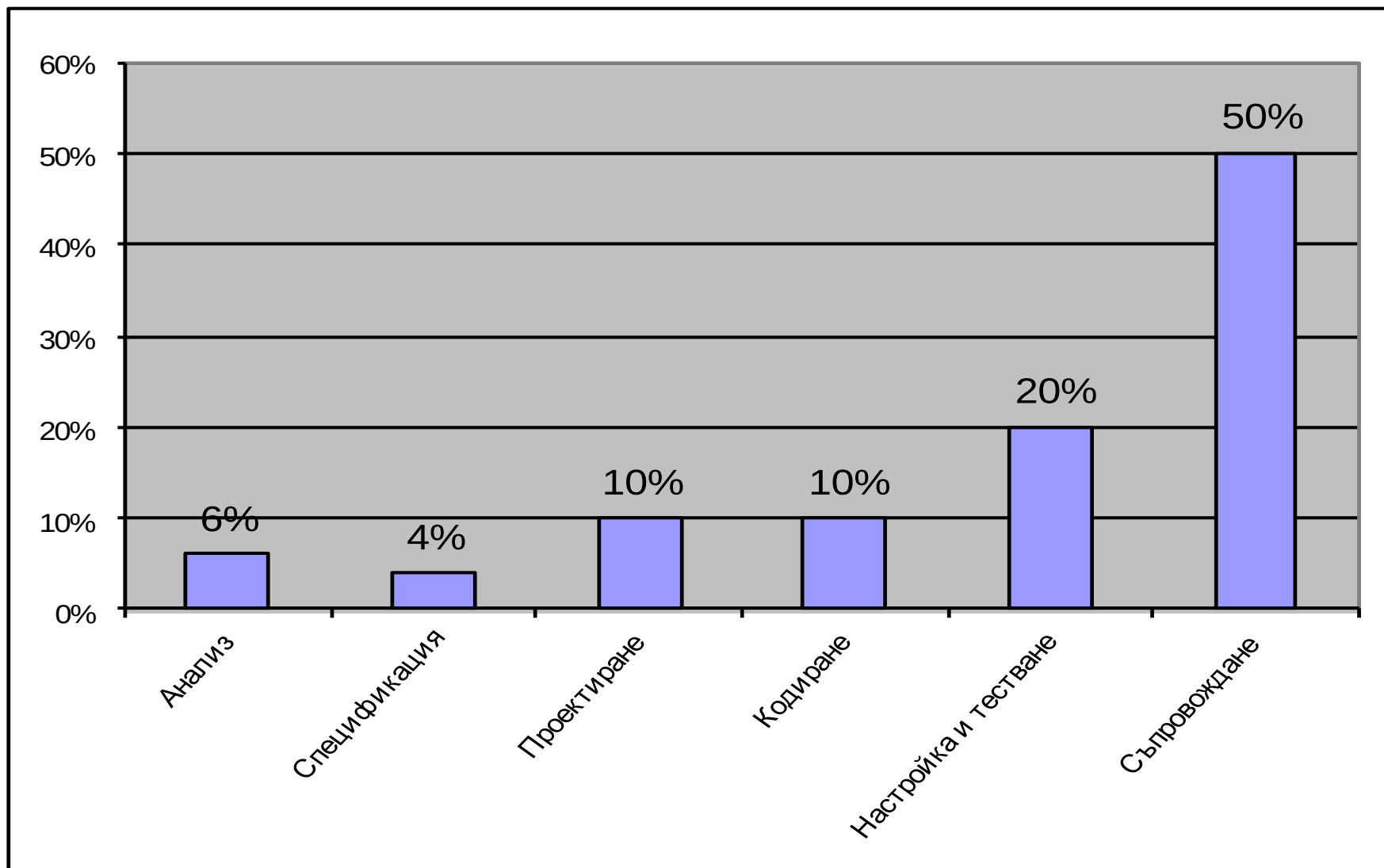
Основни фази (етапи) на Жизнения цикъл на софтуера

- **Експлоатация и съпровождане** след внедряване на системата.
 - Възможно е да се наложи **препрограмиране** на отделни модули за подобряване експлоатационните характеристики на системата.
 - Допуска се **еволюция**: променяне на софтуера като реакция на променящите се изисквания, създаване на нови версии.
 - Допуска се този етап да завърши с **формулиране на изисквания за разработка на нова система**, с което цикълът анализ, проектиране, реализация и поддръжка се затваря.

Връзки между отделните фази на ЖЦ, основните дейности и резултати към всеки един от тях



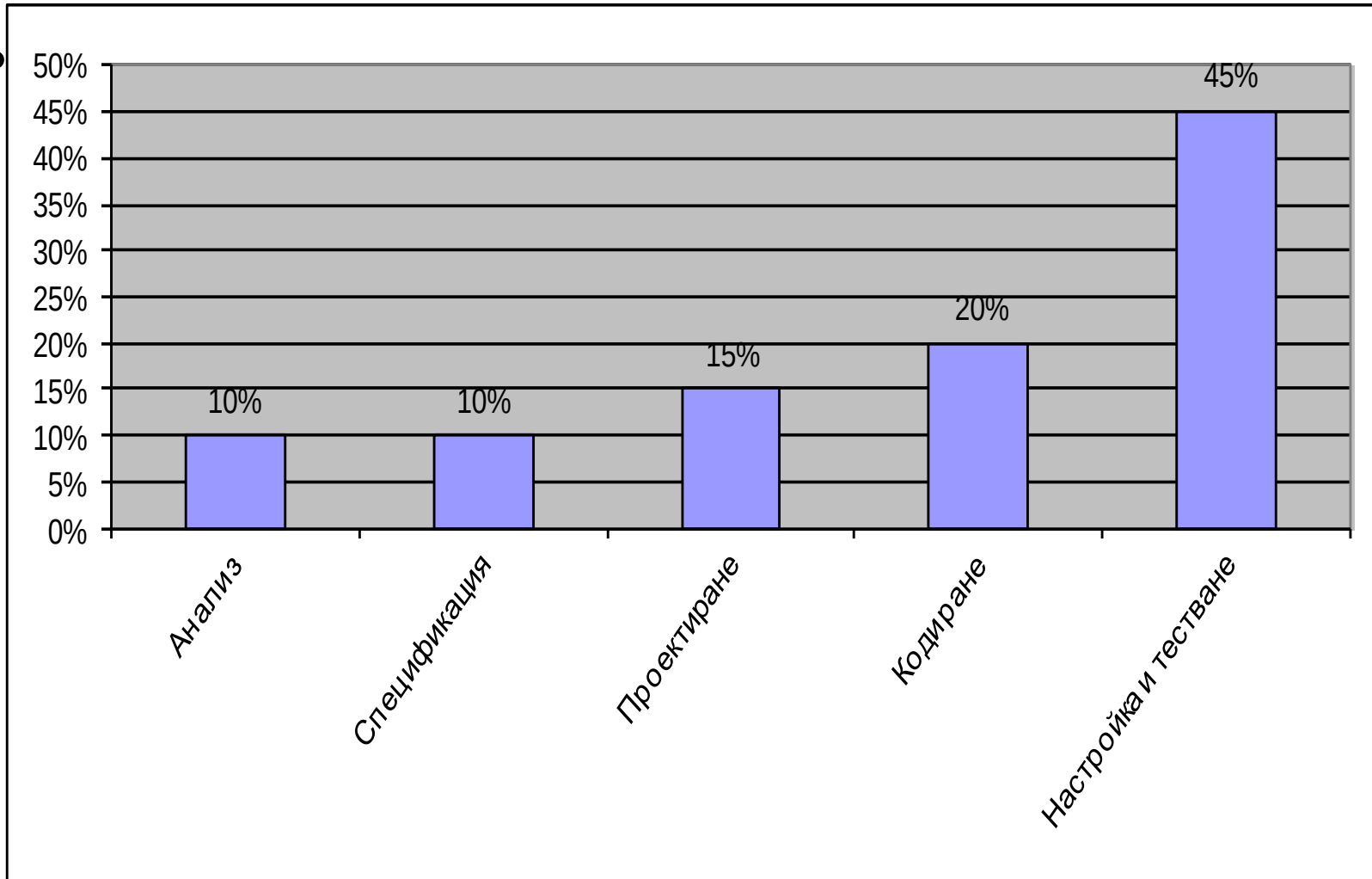
Разходи на време



Извод

- Най-голям дял при разпределението на разходите се пада на съпровождането на ПО.
- Това се отнася в най-голяма степен за такива масови продукти като операционните системи. Като правило 60 – 70% от разходите за дадена операционна система се извършват, след като системата е станала вече достъпна за потребителите, а това са разходи за съпровождане.

Разходи на средства



Извод

- Изключвайки етапа съпровождане след анализ на разходите по двете фигури по-горе се вижда, че най-трудо- и време-поглъщащ е етапът на тестване и настройка (45% от времето и 20% от средствата). Това говори както за важността на този етап, така и за необходимостта от много добротото му планиране и практическо изпълнение. В този смисъл той е един от ключовите етапи.
- Колкото корекциите се извършват на по-късни етапи, толкова тяхната цена е по-голяма. Според някои източници, ако цената на една грешка, открита при проектирането е 1 (в относителни единици), то цената на грешката, открита на етапа програмиране е 38, а при експлоатацията на системата – 114 !

Класификация на моделите на ЖЦ на ПП

Съществуват многообразие от модели на ЖЦ на ПП, на различно ниво на развитие и/или използване. Според класификацията в [1],[2] (която ние ще възприемем) те са:

1. Пълни

1.1. Едномерни

1.1.1. Хронологични (основа моделите е развитието на ПП във времето)

1.1.1.1. Стандартни (Боем, Метцгер, Фрийман)

1.1.1.2. Модифицирани (каскаден, прототипен)

1.1.1.3. Разклонени (Фокс)

1.1.2. Функционален – на Хамилтън-Зелдин

1.2. Многомерни

1.2.1. Двумерни (Гънтър)

1.2.2. Тримерни (Питърс-Трип)

1.3. Еволюционни

1.4. Спирални

2. Частични – за повторно използване (Епълтън)

Пълни модели на ЖЦ

- 1.1. Едномерни
 - 1.1.1. Хронологични (основа моделите е развитието на ПП във времето)
 - 1.1.1.1. Стандартни (Боем, Метцгер, Фрийман)
 - 1.1.1.2. Модифицирани (каскаден, прототипен)
 - 1.1.1.3. Разклонени (Фокс)
 - 1.1.2. Функционален – на Хамилтън-Зелдин
- 1.2. Многомерни
 - 1.2.1. Двумерни (Гънтър)
 - 1.2.2. Тримерни (Питърс-Трип)
- 1.3. Еволюционни
- 1.4. Спирални

Пълни Едномерни модели на ЖЦ

1.1.

Едномерни

1.1.1. Хронологични (основа моделите е развитието на ПП във времето)

1.1.1.1. Стандартни (Боем, Метцгер, Фрийман)

1.1.1.2. Модифицирани (каскаден, прототипен)

1.1.1.3. Разклонени (Фокс)

1.1.2. Функционален – на Хамилтън-Зелдин

Хронологични модели

- **Хронологични модели** – това са пълни, едномерни модели на ЖЦ, построени на хронологичен принцип.
 - Основа на тези модели е развитието на ПП линейно **последователно** във времето (затова според някои автори тези модели се наричат линейни **последователни** за разлика от итеративните).
- **Хронологични модели** се делят на:
 - 1.1.1.1. Стандартни (Боем, Метцгер, Фрийман)
 - 1.1.1.2. Модифицирани (каскаден, прототипен)
 - 1.1.1.3. Разклонени (Фокс)

Стандартни хронологични модели

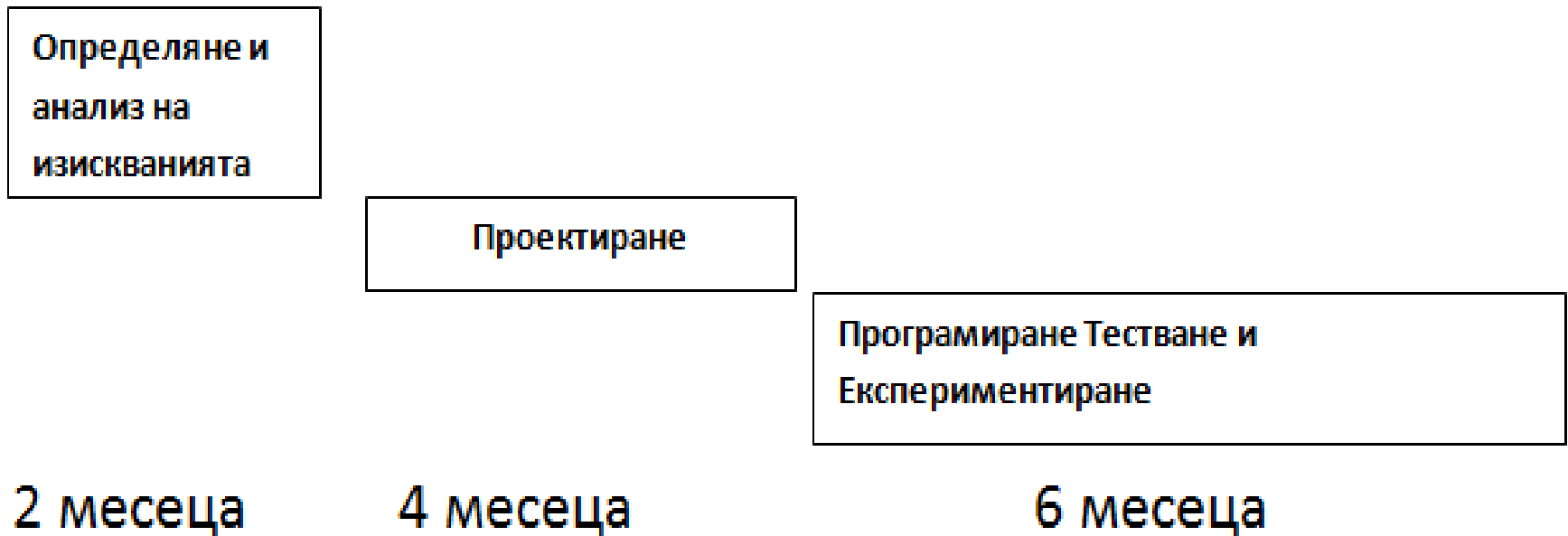
- **Стандартни хронологични модели** (на Боем, Метцгер, Фрийман):
 - Основен обект е **фазата** – отрязък от време през което се извършват определени дейности върху разработвания ПП.
 - Дейностите се изпълняват еднократно и последователно, тоест тези модели не предполагат препокриване на **фазите** и итерации.
 - Преходът от една фаза в друга става само след като бъдат завършени дейностите по текущата.
 - Всяка фаза завършва с пълен комплект документация, достатъчна за да може разработката да бъде продължена от друг екип разработчици.

Стандартни хронологични модели

Модел на Фрийман	Модел на Метцгер	Модел на Боем
1. Анализ на необходимостта от създаване на ПП 2. Специфициране на ПП: създаване и описване условията за реализацията му. Оформяне на „Спецификация на ПП“;	Определяне на софтуерната система;	1. Определяне на изискванията към ПП 2. Определяне на изискванията към средата на използване и на разработване;
1. Проектиране архитектурата на софтуерната система (външен проект) 2. Детайлно проектиране (вътрешен проект);	Проектиране;	1. Предварително проектиране 2. Детайлно проектиране;
Реализация на ПП 1. Програмиране 2. Тестване;	1. Кодиране (програмиране) 2. Вътрешно тестване (по модули и системно) 3. Приемни изпитания от независими експерти - може ли системата да се пусне на пазара;	1. Програмиране 2. Тестване и експериментиране (експериментално внедряване, проверка на работоспособността на ПП);
Поддържане: осъществяване на връзки с потребителя за инсталиране на ПП, обучение, регистриране на грешки.	Инсталиране Експлоатация на ПП.	Използване на ПП в реални условия.

Стандартни хронологични модели

Ако имаме едногодишен проект, то този проект би могъл да има следното разделяне във времето:



Стандартни хронологични модели

Положителни страни:

- На всеки етап се създава завършен набор проектна документация (**documentation-driven model**), отговаряща на критериите за пълнота и съгласуваност.
- Изпълняваните в логическа последователност фази, позволяват да се планират срокове за завършване на всички дейности и съответните разходи.

Сфера на приложение:

При създаване на ПС, за които още в началото на разработването може достатъчно точно и пълно да се формулират всички изисквания.

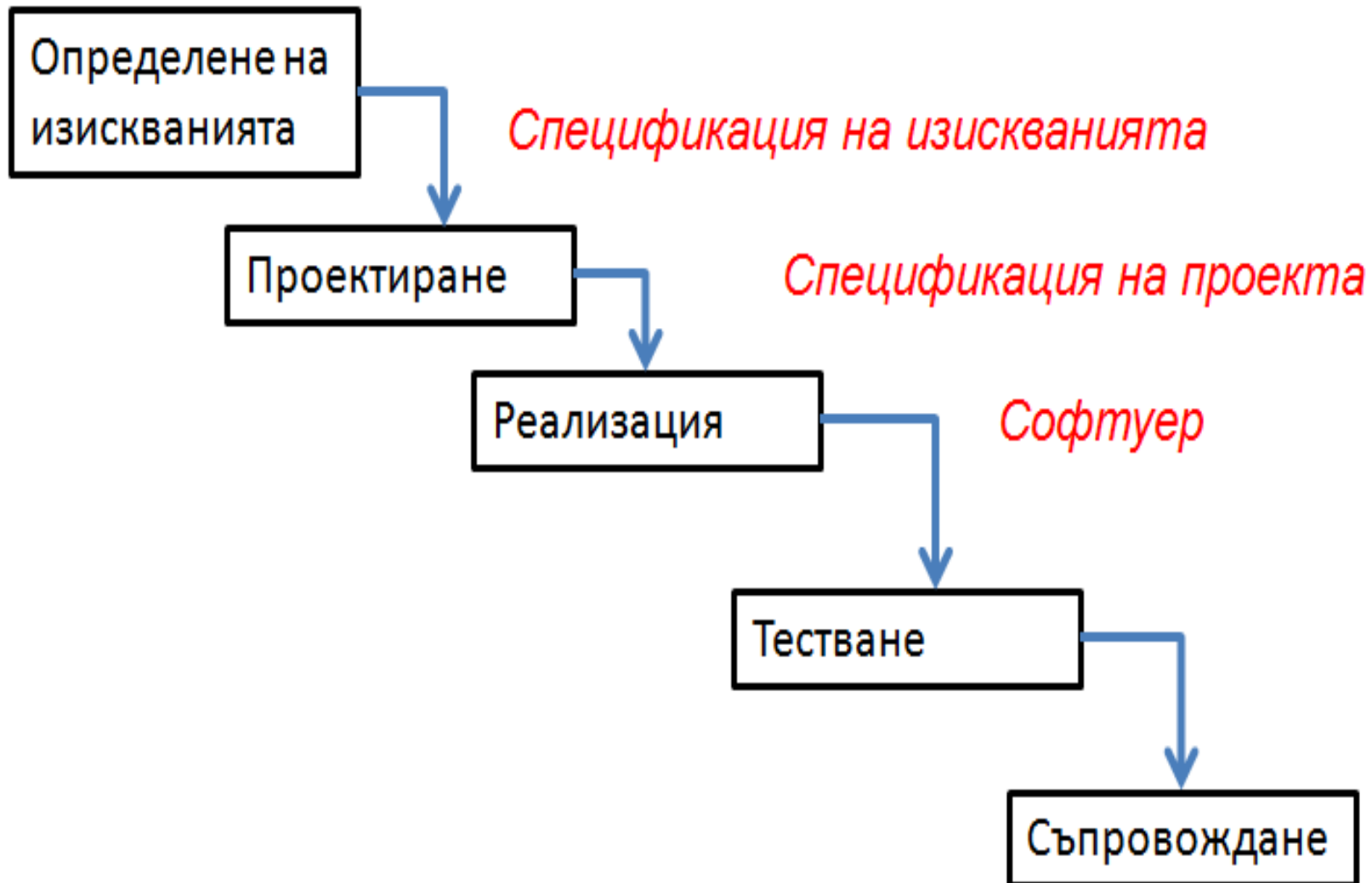
Проблеми:

- Експоненциален ръст във времето на цената на отстраняване на допуснати грешки в проектирането и разработката.

Каскаден модел (Водопаден модел - waterfall model)

- Това е модел от групата на **хронологичните, модифицирани** модели, съгласно класификацията в [1], [2]. Ще отбележим, че оригиналният водопаден модел на [Winston W. Royce](#) (1970) е не-модифициран.
- При модифицираните модели **отделните фази се припокриват във времето**. Това е една стъпка на подобрене на адекватността на модела на ЖЦ на ПП по отношение на стандартните модели.

Каскаден модел (Водопаден модел - waterfall model)



Каскаден модел (Водопаден модел - waterfall model)

- Вижда се, че тук (при модифицирания модел) има частично препокриване. Какво значи това? Отново се реализира принципа на последователно изпълнение на всяка дейност, но все пак се допуска връщане.
- Достоинство на този модел е ограничаването на възможността за връщане на повече от една стъпка назад, например не може от програмиране да се върнем на дефиниране на изискванията, но можем да се върнем на проектиране. Идеята е, че връщането катастрофално увеличава стойността на проекта и срока за изпълнение.

Каскаден модел (Водопаден модел - waterfall model)

- Този модел изключително много се прилагаше (през 70-80 години на миналия век) заради своята простота и сходност с моделите за разработване на други, не-програмни системи.
- В последствие, във връзка с развитието на Софтуерното Инженерство и осъзнаване на итеративния характер на процеса на разработване на ПО, този модел активно се критикуваше, практически, от всеки автор на статия или учебник.

Каскаден модел (Водопаден модел - waterfall model)

- Независимо от критиките, този модел се използва и досега:
 - за не-големи проекти, при които итеративността не е толкова нужна.
 - за офшорни проекти с почасово заплащане на труда.
 - като съставна част е на други модели и методологии, например V-образния модел .

Каскаден модел (Водопаден модел) - Недостатъци

- Твърде трудно е да се формулират изискванията към софтуера наведнъж (а това е изискване на този модел).
- Фазите и видовете дейности при този модел се отъждествяват. Това влече до загуба на гъвкавост при разработването.

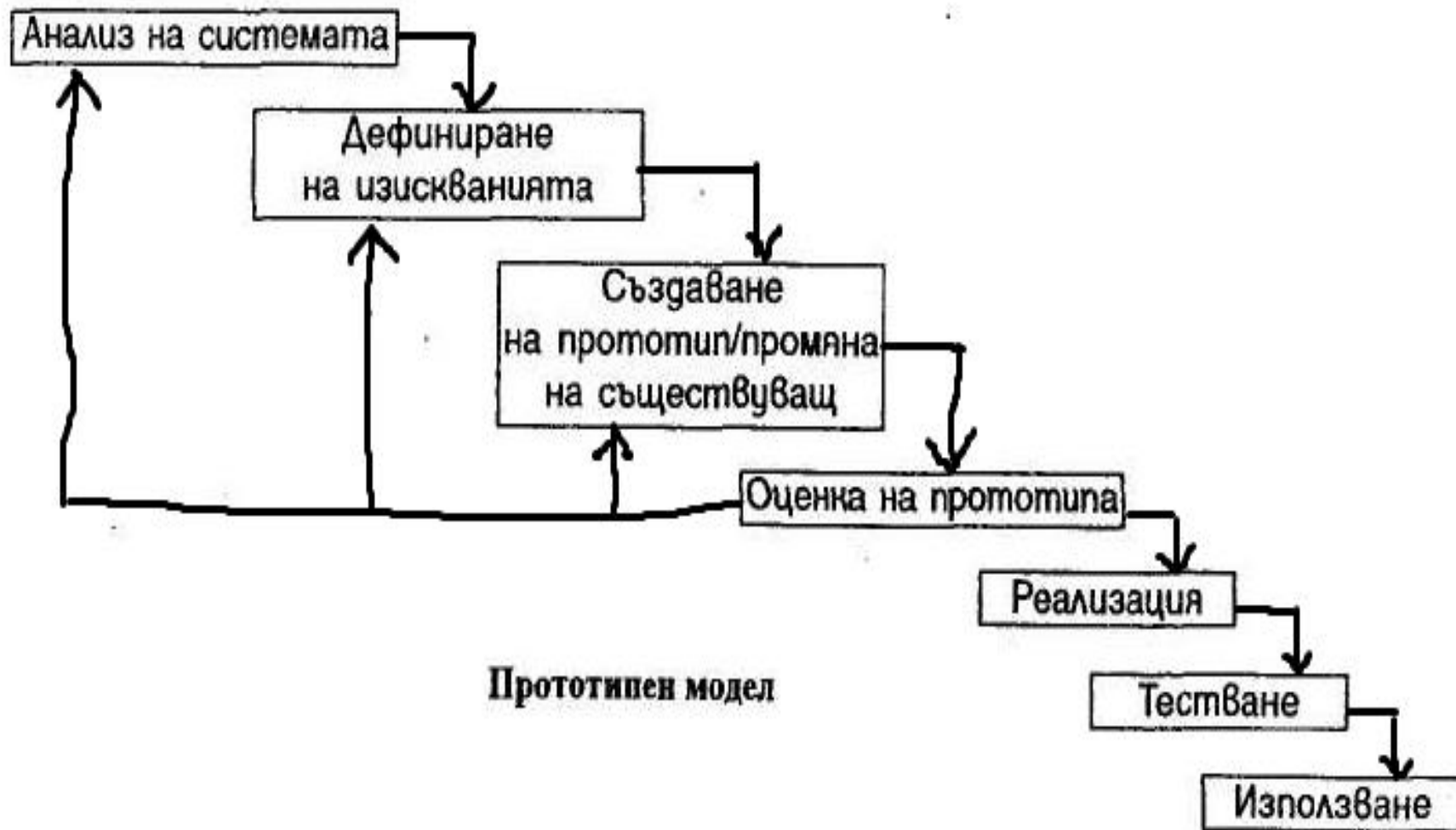
Каскаден модел (Водопаден модел) - Недостатъци

- Изисква пълно завършване на фазата=действие и описване на резултатите от нея във вид на подробен изходен документ (техническо задание, проектна спецификации); но опита показва че е невъзможно да се завърши дадена фаза напълно, например не може напълно да определим изискванията, често е невъзможно това.
- Потребителите и възложителите виждат системата едва накрая, което увеличава риска да бъде създаден продукт, който да не отговаря на очакванията на потребителите;

Прототипен модел

- Пълен хронологичен модифициран модел, който се използва твърде интензивно в практиката.
- Реализира умалена версия = прототип (т.е първи от даден тип) на крайния ПП, върху която се правят експерименти за установяване на съответствие с изисквания на потребителя. В зависимост от получените резултати се правят корекции в прототипа с различна степен на връщане към предходните стъпки.
- След окончателното доказване на правилността на функциониране на прототипа и съответствието му с изискванията за разработката се пристъпва към реализацията на крайния ПП.

Пълен Хронологичен Модифициран Прототипен модел

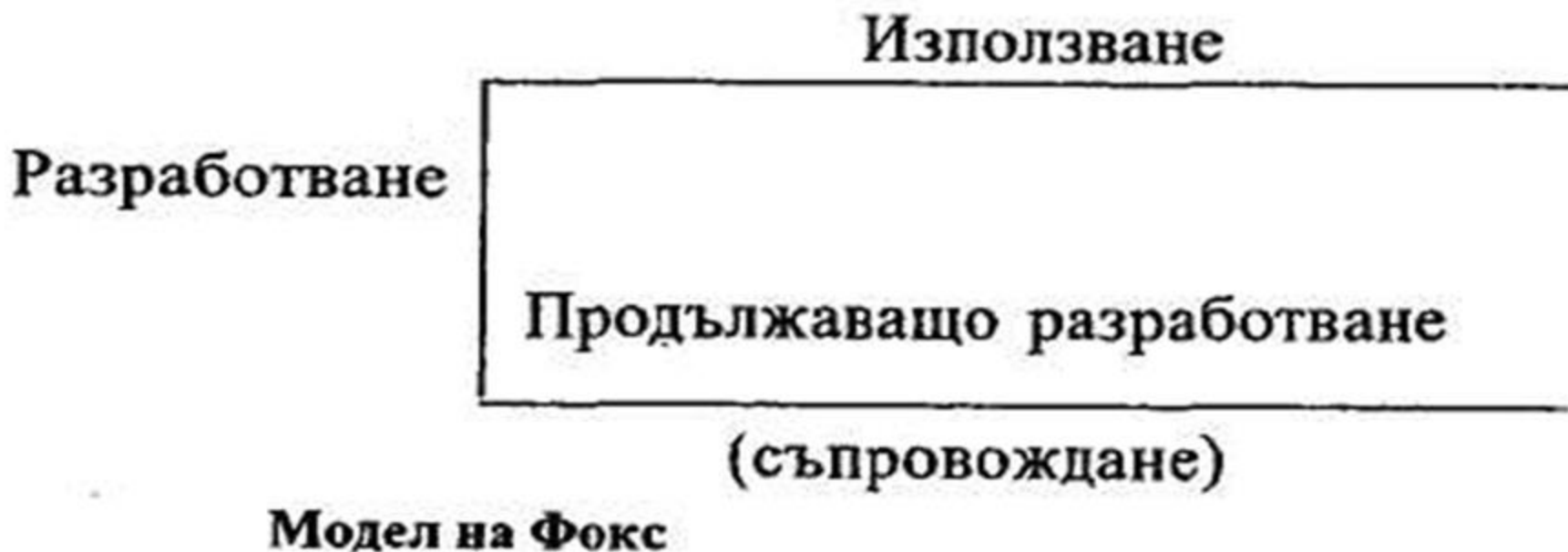


Пълен Хронологичен Модифициран Прототипен модел

- **Предимствата** на такова прототипиране са очевидни: прототипът спомага по-бързо и лесно да се изяснят изискванията на потребителя.
- **Недостатъкът** е липсата на гаранции, че прототипът ще съдържа всички съществени свойства на крайния ПП.

Разклонен модел на Фокс

- Разработен е от Фокс - известен специалист с огромен практически опит в разработването на много големи софтуерни системи.
- Включва 3 фази: разработване, използване, съпровождане:



Разклонен модел на Фокс

- Фокс твърди, че терминът „**съпровождане**“ не е подходящ и използва термина "**продължаващо разработване**".
- Съпровождането според Фокс може да отнеме при големи ПП толкова усилия, колкото и разработването.
- Като важен принос на Фокс следва да се разглежда въведената от него успоредност на двете фази: «**използване**» и «**продължаващо разработване**».

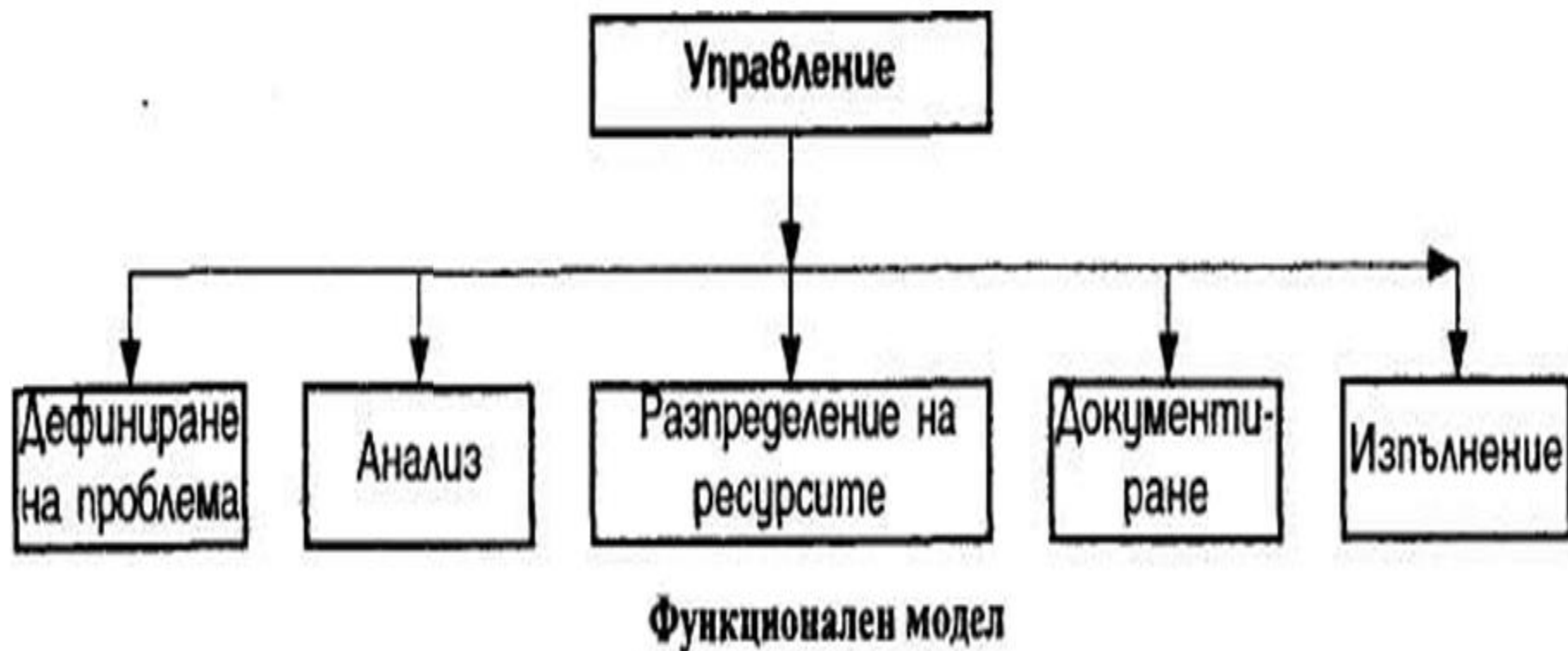
Разклонен модел на Фокс

- Дейностите за съпровождането са следните:
 - **Добавяне на нови функции**
 - **Усъвършенстване на съществуващи функции**
 - **Добавяне на функции, свързани с промени в хардуера**
 - **Отстраняване на установени грешки**

Функционален модел

- Единствен пример за функционален модел е този на Хамилтън и Зелдин (според класификацията в [1], [2]).
- Според този модел, най-същественият признак, по който следва да се декомпозира ЖЦ на ПП, е **типът извършвани дейности**.
- Група от приличащи си дейности се нарича **«функция»**.
- Следващата схема показва функциите в модела на Хамилтън и Зелдин.

Функционален модел (Хамилтън и Зелдин)



Функции на модела:

- **Дефинирането на проблема** – резултат от взаимодействието на потребителя с разработчика.
- **Анализът** представлява изясняване на проблема и избиране на решение в най-общия му вид.
- **Разпределението на ресурсите** е функция, която липсва в другите модели. Тук в нея се включва планирането и разпределението на всякакви ресурси, необходими за изпълнението на разработката (човешки, хардуерни, софтуерни, документационни и пр.).
- **Документирането** е функция, резултатът от която са различните видове документация:
 - вътрешна (за нуждите на разработката),
 - потребителска (за нуждите на експлоатацията, която се извършва от потребителя) и
 - съпровождаща (за нуждите на съпровождането).

Функции на модела и USE.IT

- **Изпълнението** е реализацията на планираните и фиксирани във вече изброените функции дейности.
- Всички функции са свързани с една **управленска функция**, която въплъщава идеята за непрекъснатата управляемост и контролируемост на процесите по разработването на програмния продукт.

Логическата връзка между отделните функции се илюстрира от стрелките и тяхната посока.

- Този модел, съгласно авторите му, се е използвал реално — на неговата основа е било разработено инструментално програмно средство под името **USE.IT**.

Пълни Многомерни модели на ЖЦ

1.2.1. Двумерни (Гънтър)

1.2.2. Тримерни (Питърс-Трип)

Модел на Гънтър

Обща характеристика

- Както се вижда, по класификацията той е в групата на **двумерните модели** — 1.2.1.
- Едното измерение е **хронологично**, другото — **функционално**. Съгласно хронологичното измерение, разработването на ПП преминава през **6 фази**. По време на тези фази с различна интензивност се осъществяват **7 функции**.
- Фазите са **Изследване, Анализ на осъществимостта, Проектиране, Програмиране, Оценка, Използване и Съпровождане**.

Модел на Гънтър

- В редовете на таблицата по-долу са представени фазите на ЖЦ на ПП, а по стълбовете — функциите, а в клетките - интензивността (+) на отделните функции по време на отделните фази:

	Изследване	Осъществимост	Проектиране	Програмиране	Изпитания	Използване
Планиране	+++	+	++	+++		
Разработване		++	+++	+++	+	
Обслужване					++	
Документиране			+	++	+++	
Изпитания					+++	
Поддържане					++	++
Съпровождане						+++

Модел на Гънтър

Тримерен модел на Питърс—Трип

- Съгласно авторите, следва да се разглеждат три измерения (аспекта) на разработването на ПП :
 - **Време** – тоест развитието във времето на ПП. Тук се отличават само **4 фази**: анализ на системата (ПП), проектиране, реализация и експлоатация.
 - **Логика** - В това измерение се категоризират **дейностите, които се извършват през отделните фази**, дефинирани вече в първото измерение.
 - **Формализъм** - В това измерение авторите включват необходимите (според тях) формални модели на ПП, а именно: **чисто мислен модел**, очевидно отразяващ първоначалните идеи на разработчика за ПП. Този модел се преобразува в **структурен**. Препоръчително е той да се базира на някакъв математически формализъм от рода на графи (евентуално някакъв фиксиран тип графи — дървета, мрежи и пр.). Последният модел е **лингвистичен** и той придобива формата на последователност от текстови оператори с определен синтаксис или семантика.
- Този модел няма практическо приложение, остава само теоретичен интерес.

Пълен Еволюционен модел

!!!Тук имаме итерации за разлика от досега
разгледаните модели

- Всички разглеждани дотук модели се базираха на вече създадени и утвърдени формални описания, наричани обикновено **спецификации**.
- Спецификациите създават яснота и определеност на разработването, но създават и трудности. Основната е, че потребителят не е в състояние от самото начало да фиксира своите изисквания.
- Прототипният модел прави известен опит за преодоляване на тази трудност, като разчита на **скициране на изискванията, вместо на строго утвърдени спецификации**.
- Моделът опитва да интегрира в едно цяло **специфициране, проектиране, разработка и валидация**.

Пълни Итерационни Модели на ЖЦ

1.3.

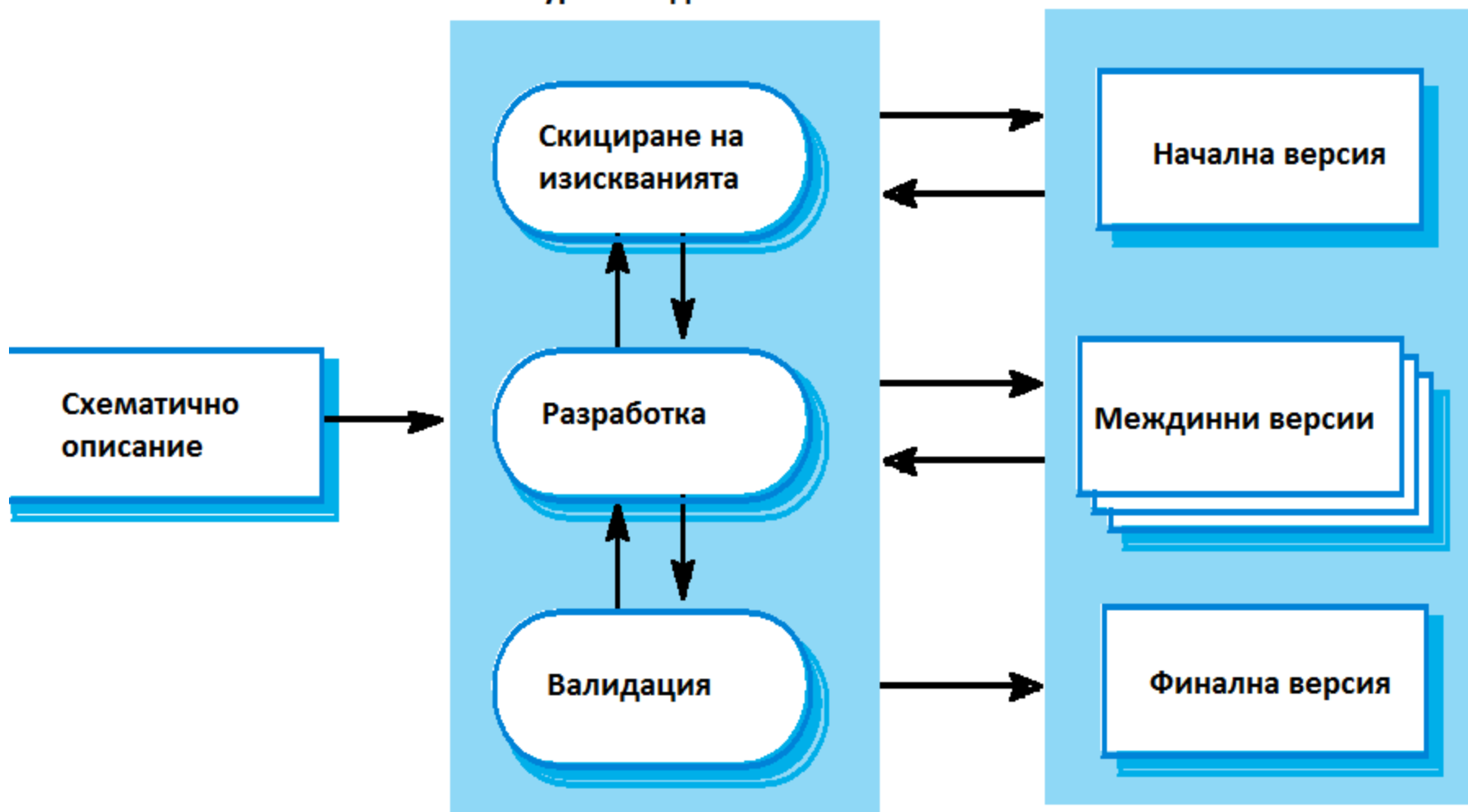
Еволюционни

1.4.

Спирални

Еволюционен модел

Конкурентни дейности



Еволюционен модел

- Конкуриращите се етапи на еволюционния модел са:
 - **Скициране на изискванията към ПП.** Тази „скица“ не е необходимо да бъде нито пълна, нито дори непротиворечива, но трябва да дава достатъчно указания на разработчиците, какво се очаква от ПП.
 - **Разработване на ПП възможно най-бързо на основата на така скицираните изисквания** (възможно най-важните модули от него).
 - **Валидация, т.е. оценяване на ПП** съвместно с потребителите или с възложителя и внасяне на изменения с оглед предявените нови изисквания към него. Това означава изменение на първоначалната функционалност на ПП и добавяне на нова, ако е необходимо.

Еволюционен модел

- Този подход обикновено се поддържа от т.н езици от **IV-то поколение** (4GL).
- Вижда се, че разработеният ПП еволюира до краен ПП, готов за доставка на потребителя.
- По същество моделът се явява итерационен, с повтарящи се етапи на специфициране и разработка до достигане на готов продукт.

Еволюционен модел

Предимства:

- Моделът е много **близък до спиралния** и в голяма степен реализира неговите предимства: **бързо води до ПП, съответстващ в най-висока степен на нуждите на потребителя.**

Недостатъци:

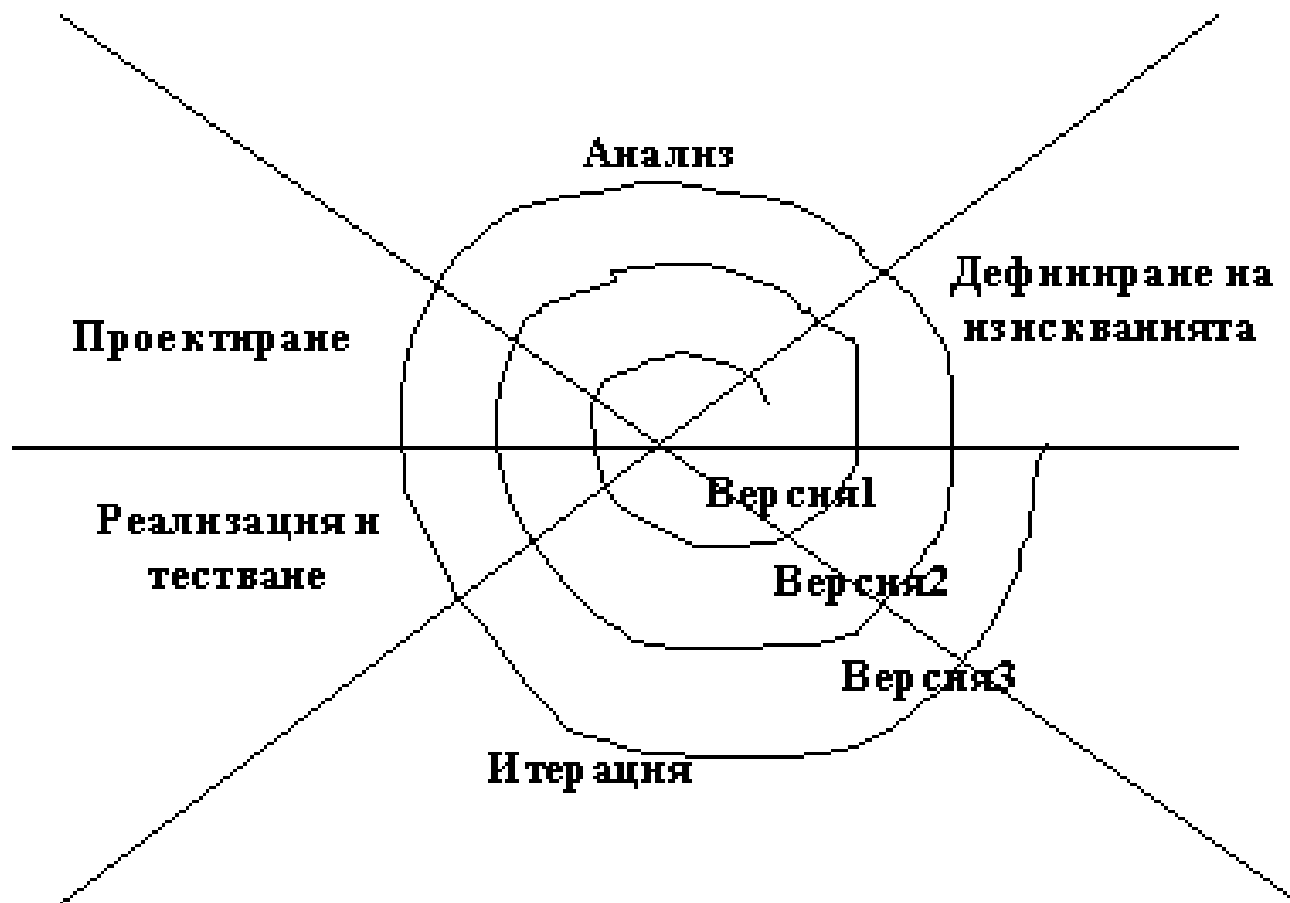
- Постоянното изменение по време на разработването **влошава структурата на ПП** до такава степен, че съпровождането на ПП може да стане изключително трудно и скъпо.
- **Цялостният процес няма желаната прозрачност** и ръководителят на проекта не е в състояние да оцени развитието му. Ето защо моделът не е удачен при големи проекти.

Спирален Модел

- Тук се залага на предварително уточнени спецификации, за разлика от еволюционния модел.
- Създаден е от Боем, през 1988, който се опитва да интегрира в еволюционен модел изискванията на управлението.
- В този модел разработването се движи спираловидно, основавайки се на спецификации. При всеки елемент на спиралата имаме:
 - Анализ на изискванията,
 - Проектиране,
 - Реализация,
 - Тестване
 - Оценка на риска и Редукция (създава се прототип). При “редукция” се създава прототип (версия на ПП), позволяващ да се редуцира неопределеността на спецификациите, да се изглади потребителски интерфейс и т.н. Това води или до обогатяване на спецификациите и постепенно - до еволюция на прототипа към последна версия на ПП.

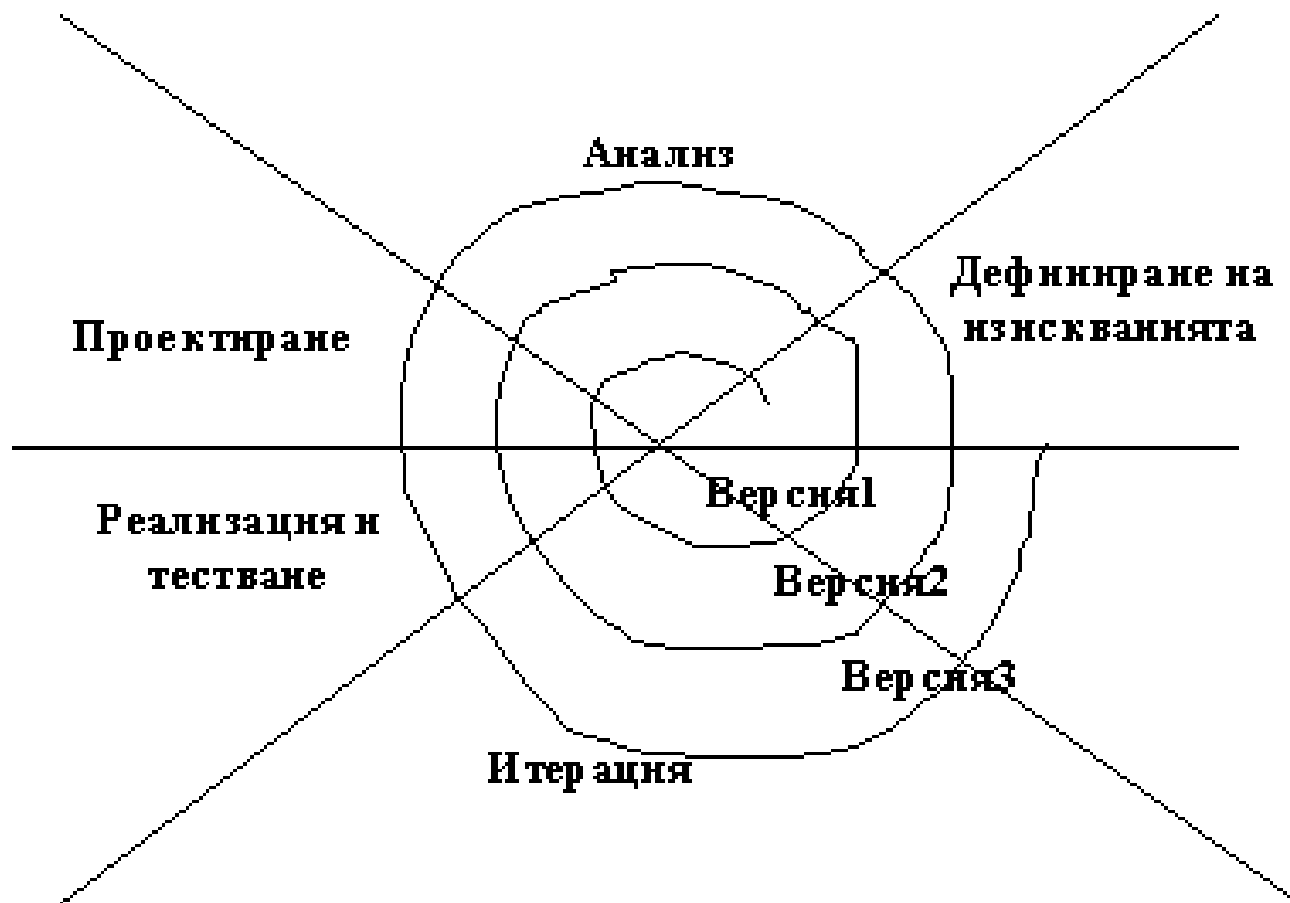
Пълен Спирален Модел

- Еволюционният и спираловидния модел са най-подходящи за разработване на интерактивни и авангардни системи, с трудно формулируеми изисквания в началния момент.



Пълнен Спирален Модел

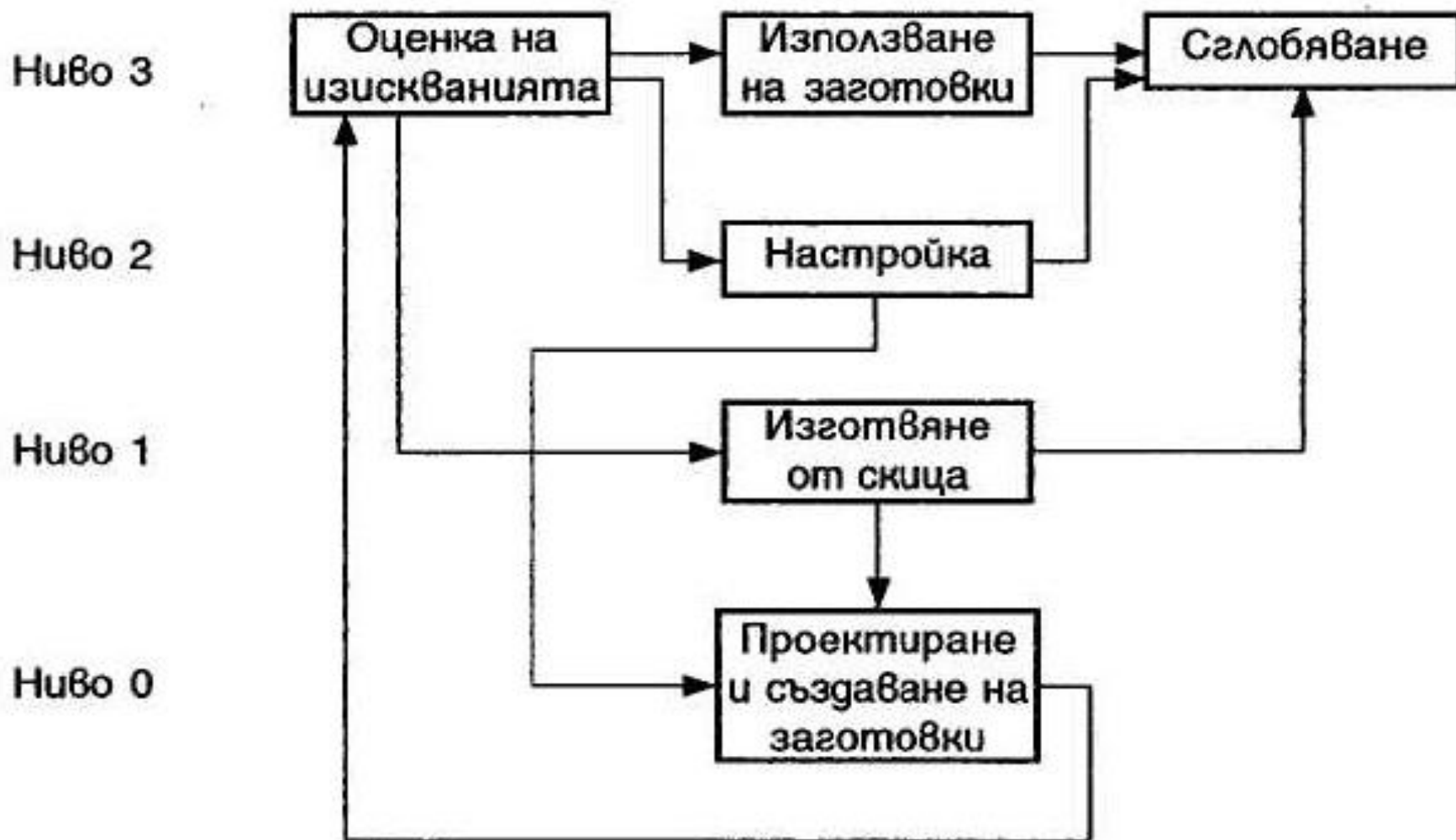
- **Предимство:** Заедно с готовия ПП са налице и добре разработени спецификации, тоест налице е документация, нещо което липсва при еволюционния модел.



Частични модели

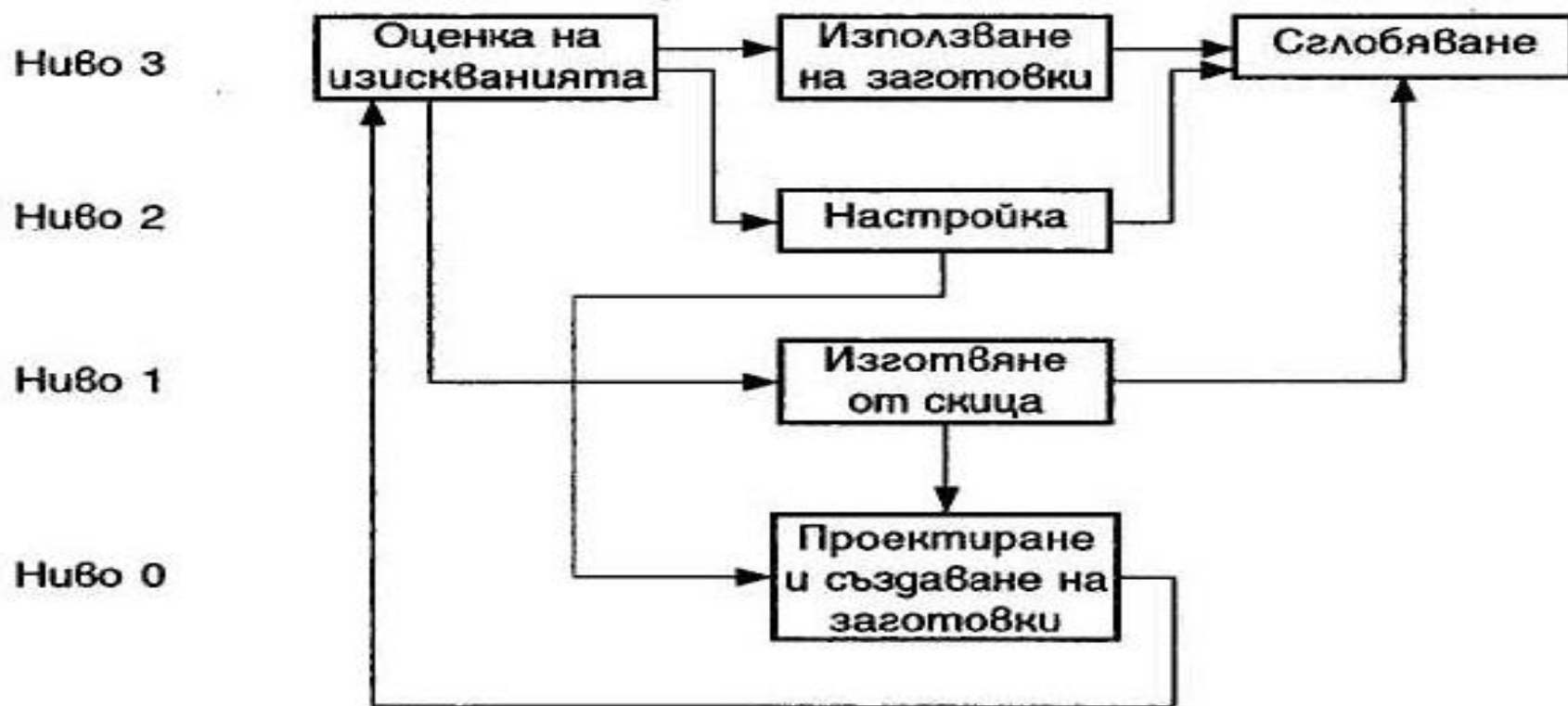
- Терминът „частични“ е използван, за да се покаже, че в този тип модели по правило липсват съществени части от естествения ЖЦ на ПП.
- Причината за това е, че тези модели се опитват да отразят т. нар. **reuse техника**, чийто смисъл е малко или повече използване на вече създадени компоненти от други ПП: програмни модули, проектантски решения, документация.
- Подходът на повторното използване се смята в момента за изключително перспективен.
- Пример за такъв модел: модел, базиран на заготовки (**assets**):

Частичен модел на заготовките



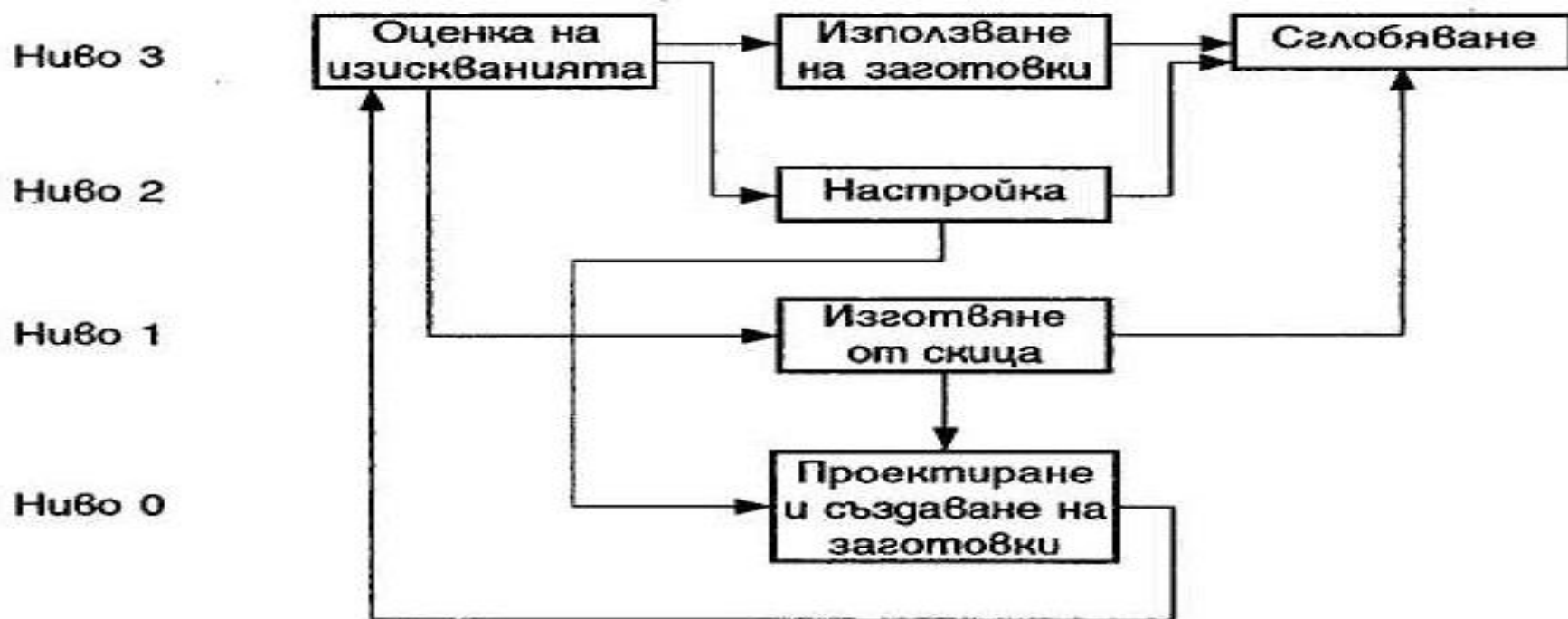
Частичен модел на "заготовките"

- Прилагането на модела на заготовките започва с **оценка на изискванията (ниво 3)**.
- В зависимост от резултата (от оценката) става насочване или към **ниво 3**, или към **ниво 2**.



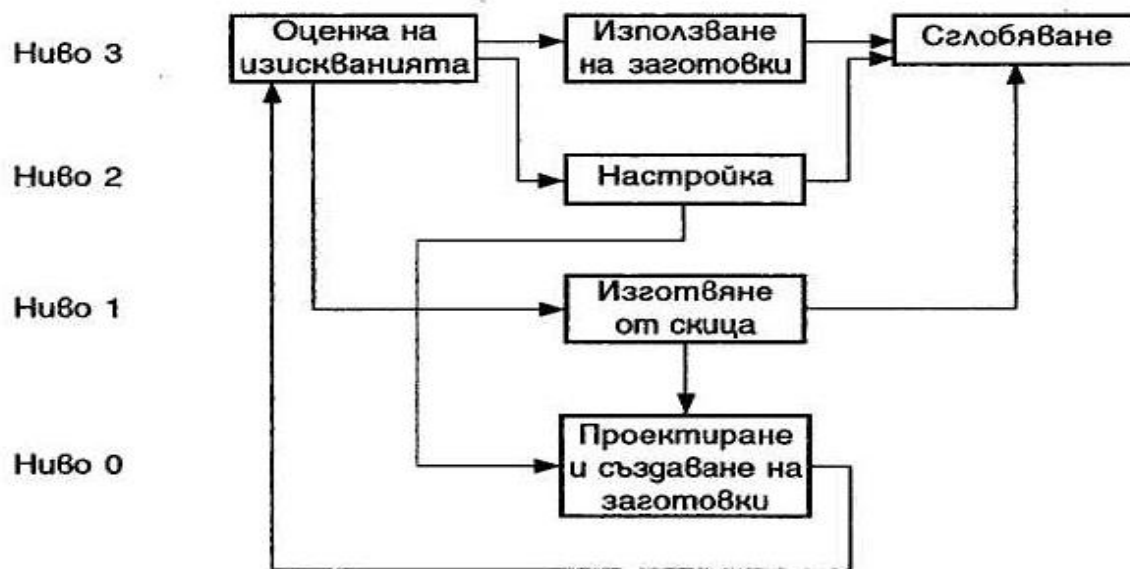
Частичен модел на "заготовките"

- Насочването към ниво 3 означава, че разработването на новия ПП е възможно практически изцяло на основата на заготовки. След подбора им следва сглобяването им и ПП е готов за разпространение.



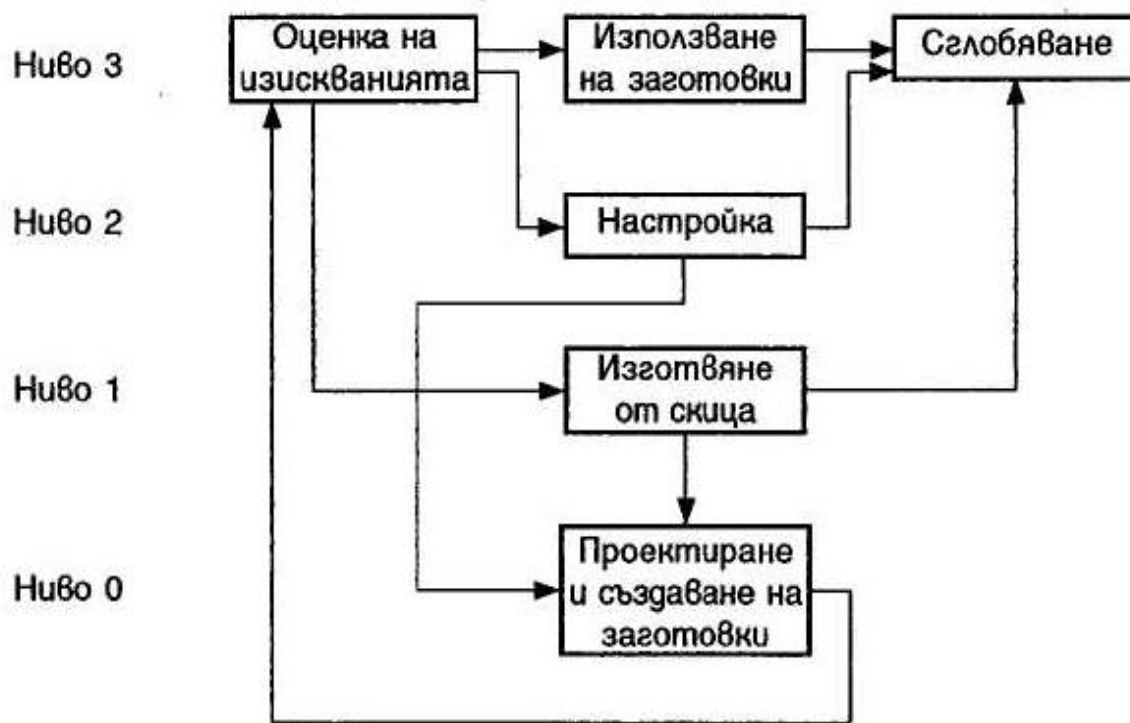
Частичен модел на "заготовките"

- Следващ по сложност е вариантът с насочване към ниво 2. Прави се настройка на избраните заготовки. На практика оттук са възможни два изхода:
- В първия случай настроените модули са готови за използване и се повтаря процедурата от ниво 3 — пристъпва се към сглобяване.
- Във втория случай има настроени модули, които въпреки настройката не удовлетворяват изискванията. Тогава се отива на ниво 0.



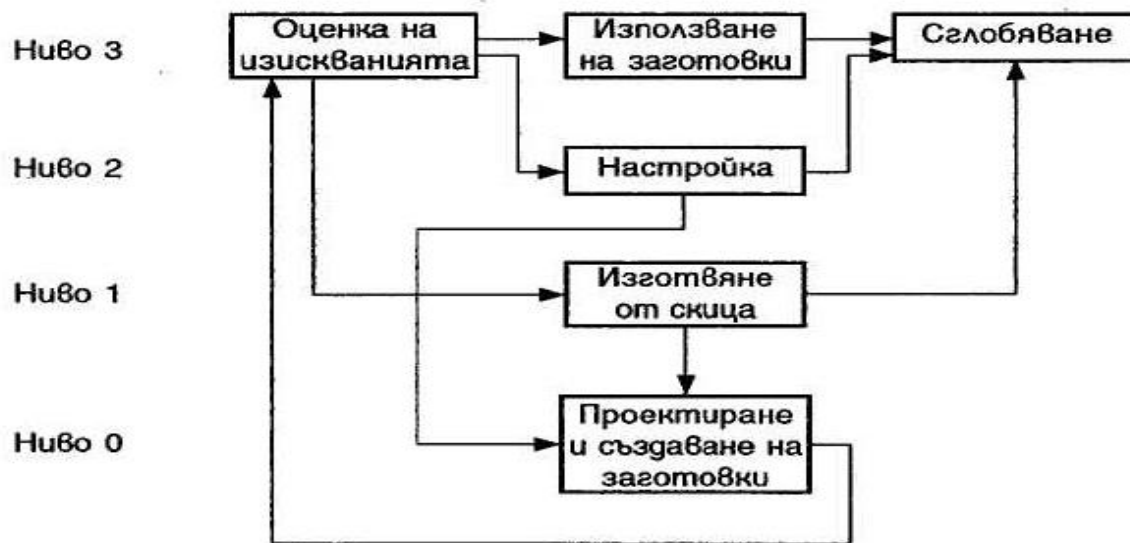
Частичен модел на "заготовките"

- Възможно е оценката на изискванията да покаже, че не е възможно насочване нито към ниво 3, нито към ниво 2. В този случай насочването е към ниво 1.
- Там разработването на необходимите модули се прави на основата на идеи от достъпните заготовки. Оттук са възможни два изхода — точно както от настройката на ниво 2.



Частичен модел на "заготовките"

- Последната възможност е ниво 0. Към процедурата на ниво 0 — проектиране и създаване на заготовки — не се отива непосредствено от оценката на изискванията. Насочването към нея е от ниво 1 или ниво 2, когато се оказва, че получените програмни модули не отговарят на изискванията.
- Ясно е, че в този модел не са отразени някои от дейностите, характерни за разработването на ПП. Идеята е да се подчертаят най-важните и специфични дейности по използването на готови елементи.



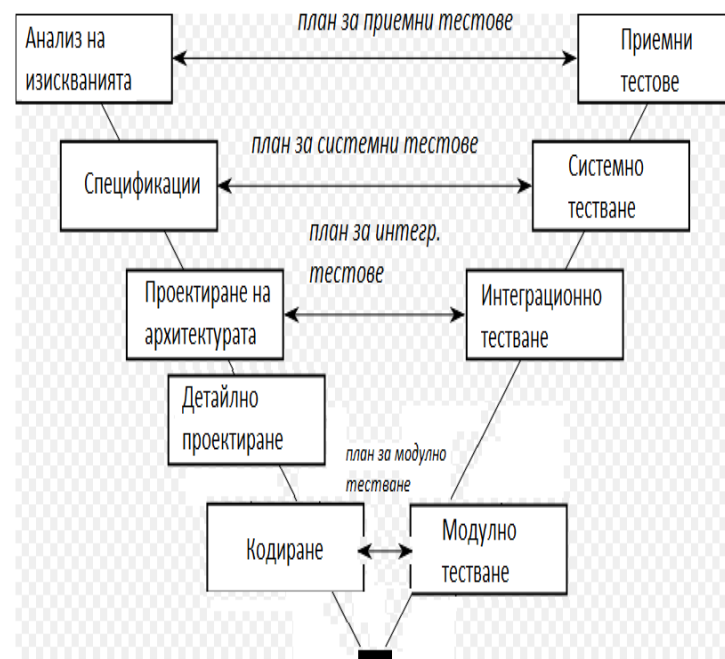
Частичен модел на "заготовките"

Някои съвременни модели на ЖЦ

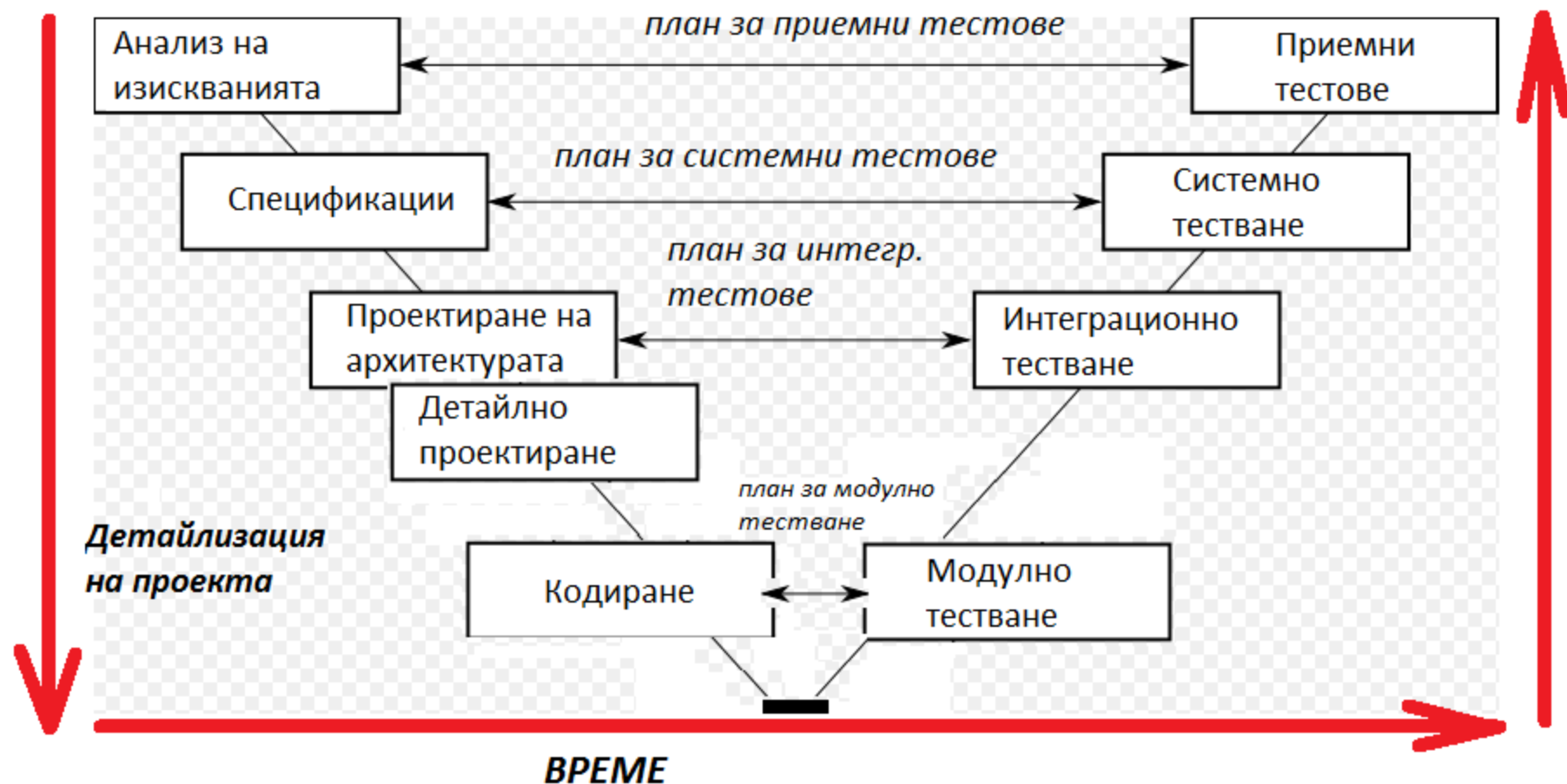
- **V-образен модел**
- **USDP (Unified Software Development Process)**

V-образен модел

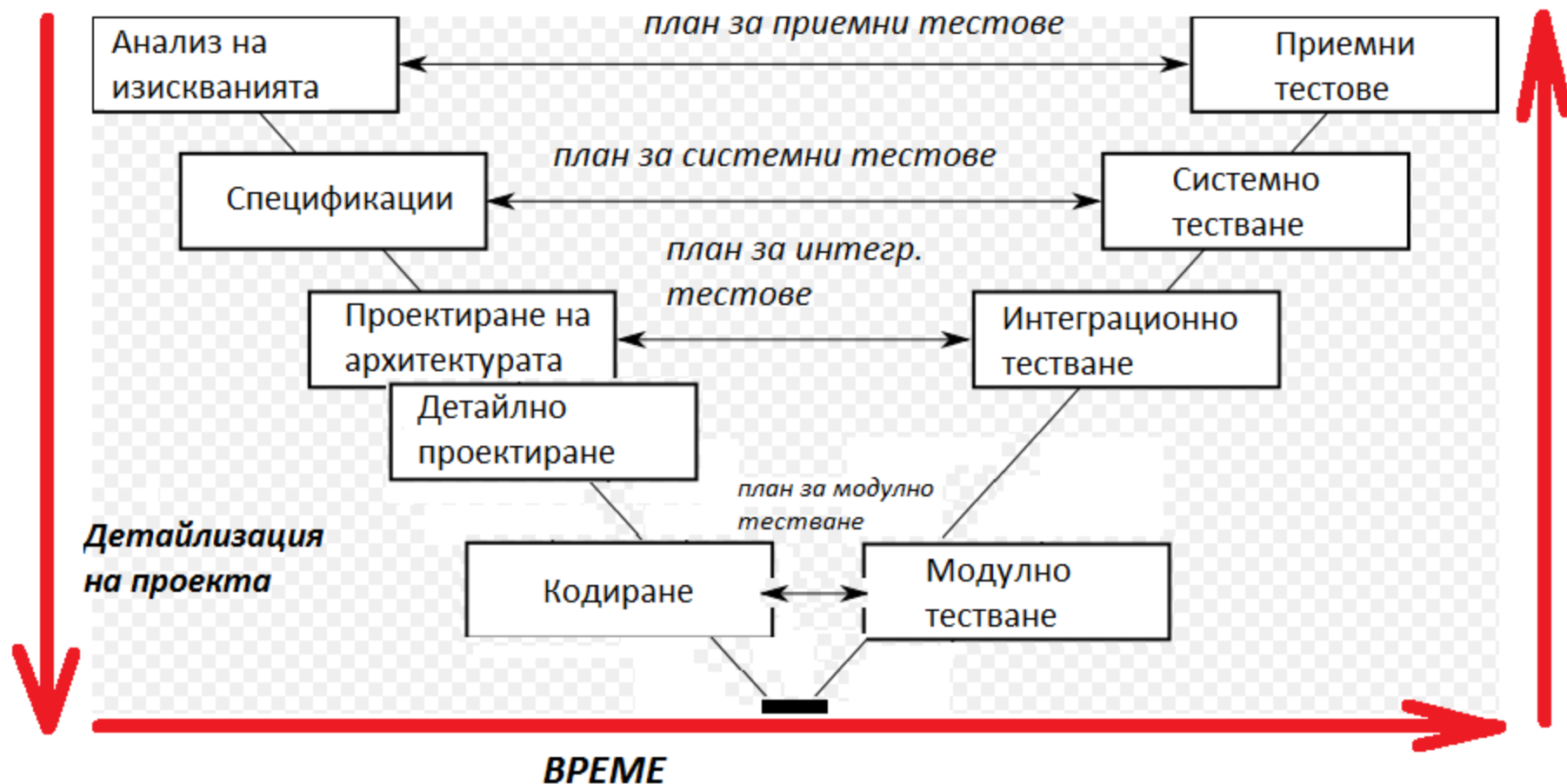
- Концепцията на V-образния модел е разработена едновременно и независимо в Германия и САЩ, в края на 1980-те години.
- Той може да бъде разглеждан като разширение на [водопадния модел](#).
- Съвременният V-Model се нарича V-Model XT, утвърден е през 2005 и е приет за стандарт за разработване на ПО на немската федерална администрация, правителство и даже за производителите на ПО в Германия.



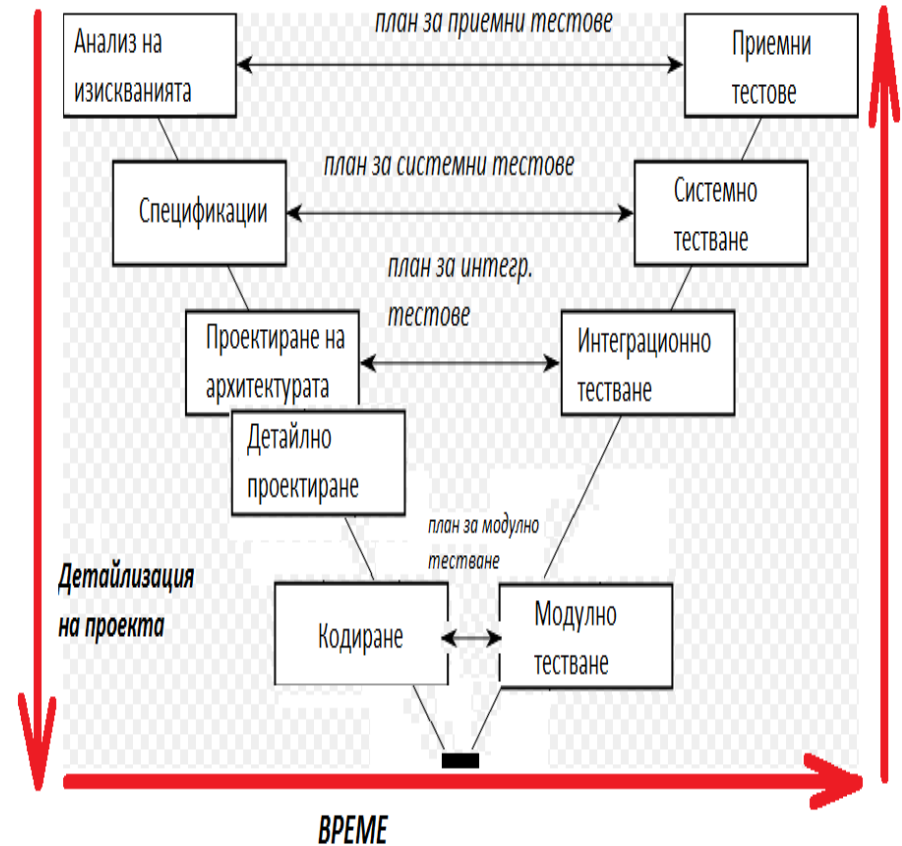
- Вместо процесът на разработване на софтуер да се движи линейно надолу, след фазата на реализация (т.е кодиране), процесите се "извиват" нагоре, като по този начин графиката на разработване придобива характерната за модела V-образна форма.



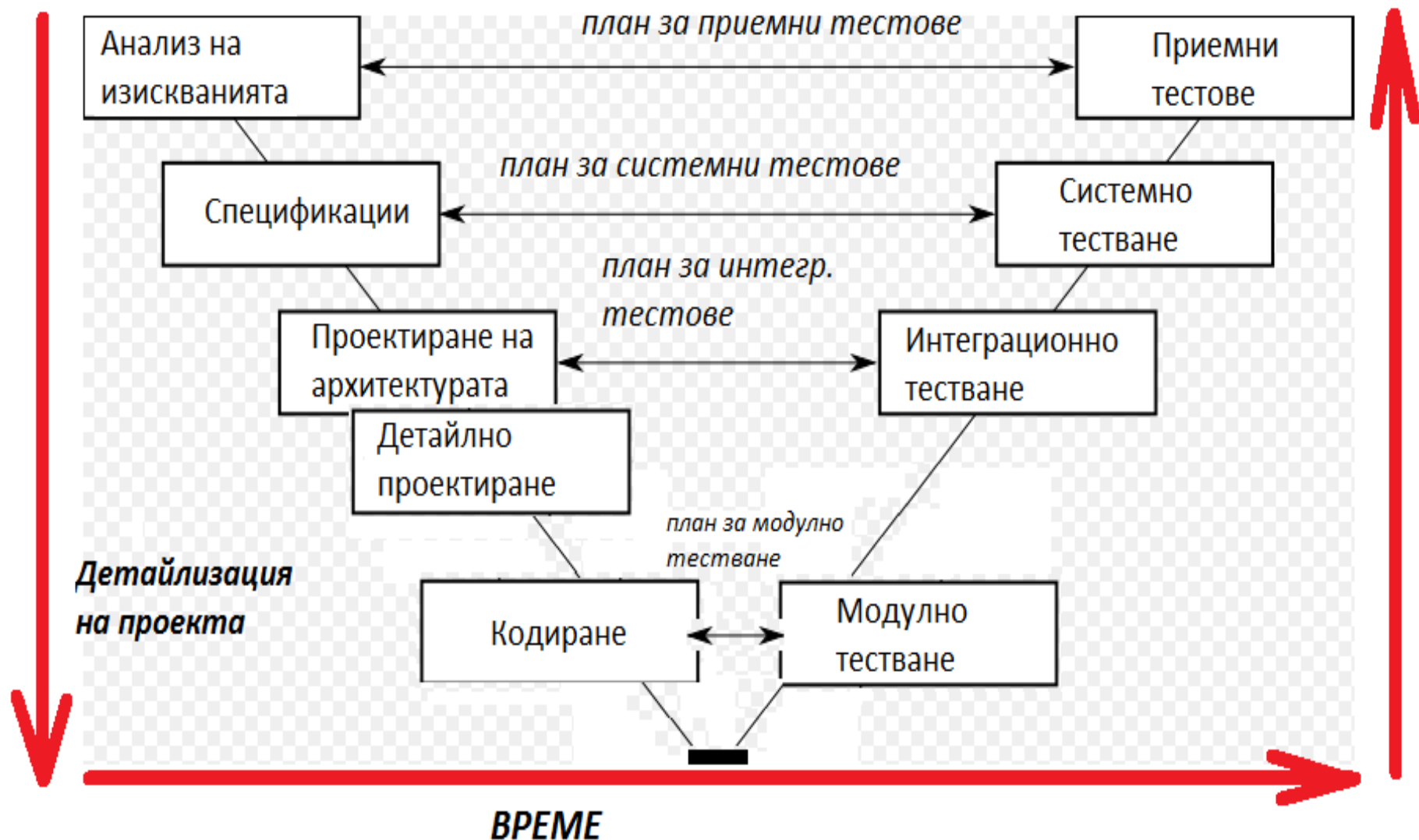
- V-моделът свързва всяка от фазите на класическия (водопаден) жизнен цикъл на софтуера със съответна фаза на тестване. Така е налице възможност за предварително планиране и проектиране на тестването.



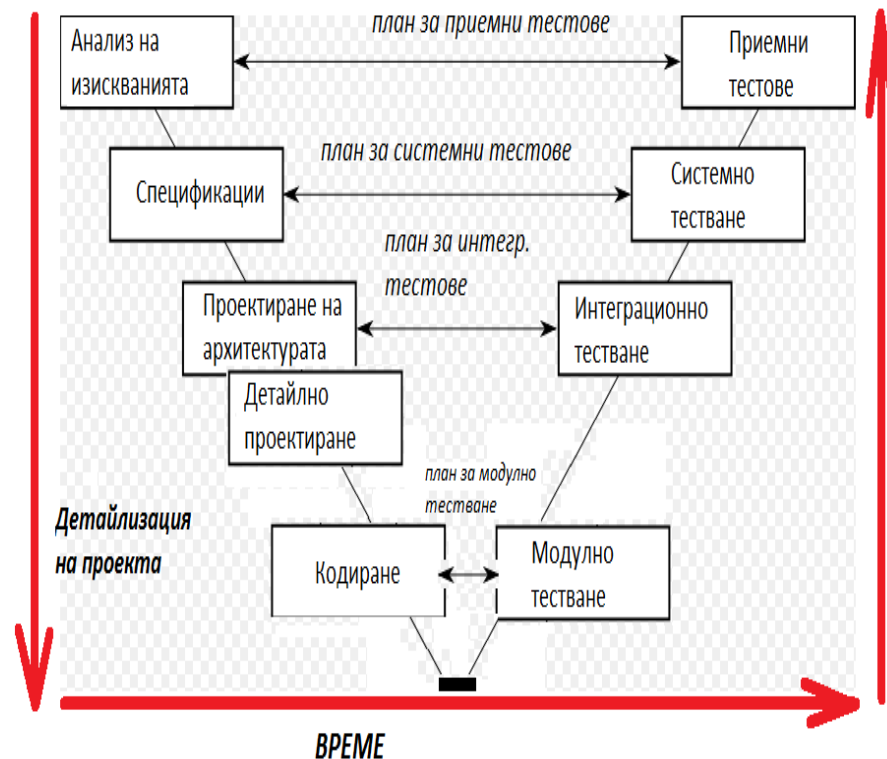
- Итерации, в проекта са възможни само хоризонтално, между лявата и дясната страна на буквата V.
- Моделът се основава на факта, че:
 - тестовите за приемане се основават предимно на изискванията,
 - тестването на системата - на изискванията и архитектурата, интеграционното тестване - на изискванията, архитектура и интерфейсите
 - компонентното тестване - на изискванията, архитектурата, интерфейси и алгоритмите.



- Разработването тече отгоре-надолу по буквата V, а тестването – отдолу нагоре по V.



- По хоризонталната ос е представено времето за разработка на проекта (заедно със степента на пълнотата на проекта), а по вертикалната – нивата на детайлизация на проекта.
- Основният принцип на V-модела е, че пълнотата на проекта се увеличава при движение от ляво на дясно, заедно с времето.



Предимства на V-образния модел:

- Това е модел, който се развива. Потребителите на V-Model-а участват в развитието и поддръжката му. Налице е «комитет по контрол на измененията на V-Model-а», който обобщава всички постъпили заявки за неговото развитие.
- V-Model-ът е универсален по отношение на типа на проекта и типа на организацията; може да бъде адаптиран за всеки проект, както и за всяка организация.

Недостатък

- Конкретен резултат може да се получи едва при достигане на долната част на буквата V.

USDP (Unified Software Development Process)

- Unified Software Development Process ("Unified Process" или UP) е **промишлен стандарт** за процес на разработка на софтуер [4], публикуван през 1995.
- Свързва се с работата на Grady Booch, Ivar Jacobson и James Rumbaugh от Rational Software Corp., сега IBM.
- UP е създаден като обобщен (generic), свободен за използване (free) процес за разработка на софтуер (т.е framework). За да бъде реално използван, той трябва да бъде придружен с подробни предписания за ползване.

Аксиоми на UP

- Съгласно UP, процесът на разработка на софтуер (ЖЦ) е:
 - “use cases driven” или “requirements driven”, т.е **движен от вариантите на използване** на създаваната система, тоест от тази функционалност, която е значима за потребителите.
 - **ориентиран към архитектурата на системата**, т.е целта е надеждна архитектура.
 - **итеративен и инкрементален** (Iterative and incremental), тоест изискванията към софтуера, архитектурата и програмните компоненти се разработват не на един дъх, а постепенно се доуточняват, в резултат на което възникват нови изисквания и нови свойства, може да се наложи и реструктуриране; по този начин се преодолява проблема със сложността, а така също и с изменчивостта на системата
 - **базиран на използване на UML** във всички фази на процеса на разработка

RUP/USDP (Rational Unified Process/Unified Software Development Process)

- RUP/USDP е вече една конкретна имплементация на UP, собственост на Rational Software Corporation, сега IBM (1999), т.е RUP/USDP е вече комерсиален продукт, тясно свързан с UP.
- RUP/USDP е «тежка категория» метод за използване на UML: съдържа множество предписания, сложна последователност от стъпки, различни роли на участниците, обхваща всички стадии на разработване на ПО.
- Неговото внедряване в една компания изисква значителни ресурси и съществена настройка на принципите на работа в нея.

Извод

- Проучването на специализираната литература показва, че продължава създаването на нови модели на ЖЦ. Поради стремежа да обхванат най-различни аспекти (технологични, организационни и икономически) новосъздаваните модели са **все по-сложни и оттам** — все по-малко приложими в практиката.
- Затова в последно време усилията са насочени към създаване и използване на **технологии или подходи за разработване на софтуер**. Всеки подход за разработване определя основните правила, практики и процедури, които трябва да се следват в процеса на създаване на ПП.
- Най-общите фундаментални подходи се наричат парадигми. Ще разграничаваме два такива подхода: традиционен (конвенционален) структурен и обектно ориентиран.
- Те са обект на разглеждане в следващи лекции

Въпроси

- Каква е целта на дисциплината „Софтуерни Технологии”?
- Какви са тенденциите в развитието на софтуера и софтуерното производство?
- Какво е „Жизнен Цикъл на Софтуера”?
- Кой са основните фази на ЖЦ на софтуера (най-общо, независимо от многообразието от съществуващи модели) и какви дейности включват? Кой са фазите, изискващи най-големи разходи, откъм време и бюджет?

Въпроси

- Кои модели на ЖЦ са хронологични? Кои са предимствата и недостатъците на стандартните хронологични модели? Каква е предимството на водопадния модел пред стандартните хронологични модели?
- Какво е характерно за V-образния модел на ЖЦ, кои са неговите предимства? Защо се твърди, че разклонения модел на ЖЦ (на Фокс) изпреварва своето време?

Основна литература:

1. Нели Манева, Аврам Ескенази, Софтуерни технологии.
Анубис София 2001
2. Нели Манева, Аврам Ескенази, Софтуерни технологии.
КЛМН, София 2006
3. Sommerville: Software Engineering ,
9. ed. Addison-Wesley, 2011,
[http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20\(intro%20txt\)%20-%20I.%20Sommerville%20\(Pearson,%202011\)%20BBS.pdf](http://neerci.ist.utl.pt/neerci_shelf/LEIC/3%20Ano/2%20Semestre/Engenharia%20de%20Software/Bibliografia/Software%20Engineering%209th%20ed%20(intro%20txt)%20-%20I.%20Sommerville%20(Pearson,%202011)%20BBS.pdf)
4. <http://gis.vsb.cz/wikivyuca/images/c/c5/Ooad2.pdf>